



DSA FINAL EXAM

COMPLETE STUDY NOTES

Data Structures & Algorithms — Full Preparation Package

Arrays

Linked Lists

Stack

Queue

Trees & BST

Graphs

DFS & BFS

Quick Sort

Merge Sort

Expressions

Syed Saad Bin Irfan

BSSE — DCS-UBIT, University of Karachi

✦ Target: 4.0 GPA | Complete Coverage | Theory + Algorithms + Dry Runs + Exam Q&A

TABLE OF CONTENTS

SESSION 1 — Arrays + Singly Linked List

Topic 1 & 2

Theory: Arrays — Properties, Algorithms, Tricks

Theory: Singly Linked List — All Insert/Delete/Search Algorithms

Exam Questions with Full Answers (Arrays + SLL)

SESSION 2 — Doubly Linked List + Stack

Topic 3 & 4

Theory: DLL — All Algorithms, Extra Pointer Steps

Theory: Stack — PUSH, POP, Overflow, Underflow

Exam Questions with Full Answers (DLL + Stack)

SESSION 3 — Queue + Trees + BST

Topic 5, 6 & 7

Theory: Queue — ENQUEUE, DEQUEUE, ISFULL, ISEMPY

Theory: Trees — Terminology, Binary Tree Types

Theory: BST — Insert, Search, Delete, Traversals

SESSION 4 — Tree Traversal + Graphs + DFS + BFS

Topic 8-11

Full Tree Traversal: Inorder, Preorder, Postorder + Expression Trees

Graphs: Types, Representations, Terminology

DFS & BFS: Algorithms, Dry Runs, Comparison

SESSION 5 & 6 — Quick Sort + Merge Sort

Topic 12 & 13

Quick Sort: Partition, Algorithm, Full Dry Runs, Complexity

Merge Sort: Merge, Algorithm, Full Dry Runs, Complexity

Exam Questions with Full Answers (Both Sorts)

SESSION 7 — Infix / Prefix / Postfix

Topic 14

Three Notations, Operator Precedence, Conversion Algorithms

Dry Runs: Infix→Postfix (4 examples), Postfix Evaluation (3 examples)

10 Full Exam Questions with Complete Answers

FINAL REVISION PACKAGE

Complete

All Algorithms Cheat Sheet — Every Structure in One Place

Master Complexity Tables (All Data Structures + Sorting)

Master Differences Tables (5 Full Comparison Tables)

35 Most Probable Exam Questions (Categorized by Probability)

Last-Minute Cheat Sheet + Exam Day Strategy

SESSION 1 OF 7

| Topics: Arrays + Singly Linked List

► TOPIC 1: ARRAYS

◆ THEORY

An **array** is a linear data structure that stores a **fixed-size, sequential collection of elements of the same data type** in **contiguous (adjacent) memory locations**.

Key Properties:

- Elements are accessed via **index** (starts at 0)
- Memory is allocated **at compile time** (static allocation)
- All elements must be of the **same data type**
- Direct/random access: Any element can be accessed in $O(1)$ using its index
- Physical address of element = Base Address + (index × size of data type)

Why Arrays in DSA? Arrays form the foundation of most data structures (stacks, queues, heaps).

Understanding array manipulation is essential before advanced structures.

◆ ALGORITHMS

Algorithm 1: TRAVERSAL — Visit and display each element one by one.

```
Algorithm: TRAVERSE
Input: Array A, size n
Output: Display all elements

Steps:
1. Start
2. For i = 0 to n-1
   → Print A[i]
3. Stop
```

Complexity: $O(n)$

💡 Memory Trick: "T for Travel — travel from 0 to n-1, visit every house."

Algorithm 2: INSERTION — Insert element X at a given position by shifting elements right.

```
Algorithm: INSERT
Input: Array A, size n, position pos, element X
Output: Updated Array

Steps:
```

```

1. Start
2. If pos < 0 OR pos > n
   → Print "Invalid Position"
3. For i = n-1 down to pos
   → A[i+1] = A[i]      ← shift right
4. A[pos] = X
5. n = n + 1
6. Stop

```

Complexity: $O(n)$

💡 Memory Trick: "Before inserting, PUSH everyone to the right. Go from the BACK (n-1) toward pos. Then place X."

⚠ Conditions to Remember:

- Invalid if pos < 0 OR pos > n (not n-1, because inserting at end is valid)
- Shift loop goes **backward**: n-1 **down to** pos (if you go forward, you overwrite data!)

Algorithm 3: DELETION — Remove element from a given position by shifting elements left.

```

Algorithm: DELETE
Input: Array A, size n, position pos
Output: Updated Array

Steps:
1. Start
2. If pos < 0 OR pos >= n
   → Print "Invalid Position"
3. For i = pos to n-2
   → A[i] = A[i+1]      ← shift left
4. n = n - 1
5. Stop

```

Complexity: $O(n)$

💡 Memory Trick: "DELETE pulls left. Start FROM pos, go forward to n-2. Each element covers the one before it."

⚠ Key Difference from Insertion:

- Invalid if pos >= n (not > n)
- Loop goes **forward**: pos **to** n-2
- **n decreases** (n = n-1); insertion increases (n = n+1)

Algorithm 4: LINEAR SEARCH — Search for element X and return its index.

```

Algorithm: SEARCH
Input: Array A, size n, element X
Output: Index of X or -1

Steps:
1. Start
2. For i = 0 to n-1
   → If A[i] == X

```

```
→ Return i
3. Return -1
4. Stop
```

Complexity: $O(n)$

💡 Memory Trick: "Check each door (index). If the person (X) is there, return the door number. If no door matches, return -1."

◆ EXAM QUESTIONS — ARRAYS

Q1. What is an array? List its properties and explain with example.

An array is a linear data structure that stores elements of the **same data type** in **contiguous memory locations**, accessed via a numeric index starting from 0.

Properties:

- Fixed size determined at compile time
- Random access in $O(1)$ time using index
- Index-based addressing: $\text{Address} = \text{Base} + (i \times \text{datatype size})$
- Homogeneous elements (same type)

Example: Array A = [10, 20, 30, 40, 50]

$A[0] = 10, A[1] = 20, A[4] = 50$

If base address = 1000, int size = 2 bytes $\rightarrow A[3] = 1000 + (3 \times 2) = 1006$

Q2. Write and explain the Insertion Algorithm for arrays. What happens when you insert 25 at position 2 in [10, 20, 30, 40]?

(Write full INSERT algorithm as above)

Dry Run: Array: [10, 20, 30, 40], $n=4$, $\text{pos}=2$, $X=25$

- Check: $\text{pos}=2$, not < 0 or $> 4 \rightarrow$ Valid
- $i=3$: $A[4] = A[3] = 40 \rightarrow [10, 20, 30, 40, 40]$
- $i=2$: $A[3] = A[2] = 30 \rightarrow [10, 20, 30, 30, 40]$
- $A[2] = 25 \rightarrow [10, 20, 25, 30, 40]$
- $n = 5$

Result: [10, 20, 25, 30, 40]

Q3. Differentiate between Insertion and Deletion in arrays.

Feature	Insertion	Deletion
Direction of shift	Right (backward loop)	Left (forward loop)
Loop range	$n-1$ down to pos	pos to $n-2$

Size change	$n = n+1$	$n = n-1$
Invalid condition	$\text{pos} < 0 \text{ OR } \text{pos} > n$	$\text{pos} < 0 \text{ OR } \text{pos} \geq n$
Complexity	$O(n)$	$O(n)$

Q4. What is the time complexity of array operations and why?

- **Traversal:** $O(n)$ — visits every element once
- **Insertion:** $O(n)$ — shifting n elements in worst case
- **Deletion:** $O(n)$ — shifting n elements in worst case
- **Search (Linear):** $O(n)$ — may check every element
- **Access by index:** $O(1)$ — direct memory calculation, no looping

🕒 EXAM STRATEGY — ARRAYS

1. Always write the algorithm in **numbered steps** as shown
2. When doing dry run, show **each step of the loop**
3. Mention **Overflow/Underflow conditions** — examiners love this
4. State complexity at the end of every algorithm
5. For differences, always use a **table format** — looks professional and scores full marks

► TOPIC 2: SINGLY LINKED LIST (SLL)

◆ THEORY

A **Singly Linked List (SLL)** is a dynamic linear data structure consisting of **nodes** where each node has:

1. **Data** — stores the value
2. **Next pointer** — stores the address of the next node

The last node's **Next = NULL** (marks end of list). The **HEAD pointer** points to the first node.

```
[10 | *] → [20 | *] → [30 | NULL]
HEAD
```

Why Linked List over Array?

- Dynamic size (grows/shrinks at runtime)
- Efficient insertion/deletion (no shifting)
- Memory allocated as needed (no wastage)
- **Disadvantage:** No random access; must traverse from HEAD

◆ INSERTION ALGORITHMS

1. Insertion at Beginning

```
Algorithm: INSERT_BEGIN
Input: Element X
Output: Updated List
```

```

Steps:
1. Start
2. Create new node N
3. Set N.data = X
4. Set N.next = HEAD      ← new node points to old HEAD
5. Set HEAD = N           ← HEAD now points to new node
6. Stop

```

Complexity: $O(1)$

💡 Trick: "New node enters from the front. It grabs HEAD's target, then becomes the new HEAD."

2. Insertion at End

```

Algorithm: INSERT_END
Input: Element X
Output: Updated List

Steps:
1. Start
2. Create new node N, set N.data = X, N.next = NULL
3. If HEAD == NULL
    → Set HEAD = N, Stop      ← empty list case
4. Set TEMP = HEAD
5. While TEMP.next != NULL
    → TEMP = TEMP.next        ← reach last node
6. Set TEMP.next = N
7. Stop

```

Complexity: $O(n)$

💡 Trick: "Walk to the last node (TEMP.next == NULL), then attach N there."

⚠ Must handle: Empty list (HEAD == NULL) separately!

3. Insertion at Position (Middle)

```

Algorithm: INSERT_POS
Input: Element X, Position pos
Output: Updated List

Steps:
1. Start
2. Create new node N, set N.data = X
3. If pos == 0
    → Insert at beginning (use INSERT_BEGIN)
4. Set TEMP = HEAD
5. Move TEMP to node at (pos-1)
6. Set N.next = TEMP.next    ← N points to next node
7. Set TEMP.next = N         ← previous node points to N
8. Stop

```


Complexity: $O(n)$

💡 Trick: "Stop at pos-1 (one step before). Wire N between pos-1 and pos."
Order matters: Set N.next FIRST, then update TEMP.next. Reverse = data loss!

◆ DELETION ALGORITHMS

1. Deletion from Beginning

```
Algorithm: DELETE_BEGIN
Output: Updated List

Steps:
1. Start
2. If HEAD == NULL → Print "Underflow", Stop
3. Set TEMP = HEAD
4. Set HEAD = HEAD.next      ← move HEAD forward
5. Delete TEMP               ← free old HEAD
6. Stop
```

Complexity: $O(1)$

💡 Trick: "Save old HEAD in TEMP, move HEAD forward, delete TEMP."

2. Deletion from End

```
Algorithm: DELETE_END
Output: Updated List

Steps:
1. Start
2. If HEAD == NULL → Print "Underflow", Stop
3. If HEAD.next == NULL
   → Set HEAD = NULL, Stop ← only one node
4. Set TEMP = HEAD
5. While TEMP.next.next != NULL
   → TEMP = TEMP.next      ← stop at second-to-last
6. Delete TEMP.next
7. Set TEMP.next = NULL
8. Stop
```

Complexity: $O(n)$

💡 Trick: "Walk until TEMP.next.next == NULL (two steps from end). Then cut TEMP.next."
Why TEMP.next.next? Because you need TEMP at the **second-to-last** node to set it to NULL.

3. Deletion at Position

```
Algorithm: DELETE_POS
Input: Position pos
Output: Updated List
```

```

Steps:
1. Start
2. If HEAD == NULL → Print "Underflow", Stop
3. If pos == 0
    → Delete from beginning
4. Set TEMP = HEAD
5. Move TEMP to node at (pos-1)
6. Set DEL = TEMP.next      ← node to delete
7. Set TEMP.next = DEL.next ← bypass DEL
8. Delete DEL
9. Stop

```

Complexity: $O(n)$

💡 Trick: "Stop at pos-1. Store pos node in DEL. Bridge over it (TEMP.next = DEL.next). Delete DEL."

◆ SEARCHING ALGORITHM

```

Algorithm: SEARCH
Input: Element X
Output: Position or "Not Found"

Steps:
1. Start
2. Set TEMP = HEAD, pos = 0
3. While TEMP != NULL
    If TEMP.data == X
        → Return pos
    TEMP = TEMP.next
    pos = pos + 1
4. Return "Not Found"
5. Stop

```

Complexity: $O(n)$

◆ COMPLEXITY SUMMARY TABLE

Operation	Complexity
Insertion at Beginning	$O(1)$
Insertion at End	$O(n)$
Insertion at Position	$O(n)$
Deletion from Beginning	$O(1)$
Deletion from End	$O(n)$
Deletion at Position	$O(n)$
Searching	$O(n)$

💡 Why Beginning is $O(1)$? Because you only update HEAD — no traversal needed!

◆ EXAM QUESTIONS — SINGLY LINKED LIST

Q1. What is a Singly Linked List? How does it differ from an Array?

A Singly Linked List is a dynamic data structure of nodes, each having **data** and a **next pointer** to the next node. The last node points to NULL.

Feature	Array	Singly Linked List
Memory	Contiguous	Non-contiguous
Size	Fixed (static)	Dynamic
Access	O(1) random	O(n) sequential
Insertion/Deletion	O(n) due to shifting	O(1) at beginning
Memory usage	May waste space	Allocates as needed

Q2. Explain insertion at end in SLL with a dry run. Insert 40 at end of 10→20→30.

(Write INSERT_END algorithm)

Dry Run: List: 10→20→30→NULL, Insert 40

- Create N, N.data=40, N.next=NULL
- HEAD != NULL, so proceed
- TEMP = HEAD (points to 10)
- TEMP.next = 20 (not NULL) → TEMP = 20
- TEMP.next = 30 (not NULL) → TEMP = 30
- TEMP.next = NULL → EXIT loop
- TEMP.next = N
- **Result: 10→20→30→40→NULL ✓**

Q3. Why do we use TEMP.next.next in deletion from end? Why not TEMP.next?

In deletion from end, we need to reach the **second-to-last node**, not the last. Because:

- We need to set the second-to-last node's **next = NULL**
- If we stop at TEMP.next == NULL, TEMP is the last node, and we can't update the previous node's pointer
- By using TEMP.next.next == NULL, TEMP stops at the second-to-last node, allowing us to delete TEMP.next and set TEMP.next = NULL properly

Q4. What is the difference between Deletion from Beginning and Deletion from End in SLL?

Feature

Delete End

Delete Beginning		
Complexity	O(1)	O(n)
Traversal needed	No	Yes
HEAD updated	Yes (HEAD = HEAD.next)	No (TEMP.next = NULL)
Condition checked	HEAD == NULL	TEMP.next.next == NULL
Special case	None	Single node check

Q5. Trace the SEARCH algorithm to find 30 in list: 10→20→30→40

- TEMP = 10, pos = 0 → 10 ≠ 30 → move
- TEMP = 20, pos = 1 → 20 ≠ 30 → move
- TEMP = 30, pos = 2 → 30 == 30 → **Return pos = 2** ✓

🕒 EXAM STRATEGY — SINGLY LINKED LIST

1. Always draw the linked list **visually** (boxes with arrows) before writing the algorithm in dry runs
2. Always mention the **underflow check** (HEAD == NULL) at the start of every deletion
3. For insertion at position — always state "Set N.next before TEMP.next" and explain WHY
4. Complexity must be written with explanation
5. Special cases (empty list, single node, position 0) score extra marks — always mention them

🔑 MASTER MEMORY TRICKS SUMMARY — SESSION 1

Algorithm	Core Trick
Array Insert	Go BACKWARD (n-1 to pos), shift right, then place
Array Delete	Go FORWARD (pos to n-2), shift left
SLL Insert Begin	N.next = HEAD first, then HEAD = N
SLL Insert End	Walk till TEMP.next == NULL, then link
SLL Insert Mid	Stop at pos-1, N.next = TEMP.next, TEMP.next = N
SLL Delete Begin	Save HEAD, move HEAD forward, delete saved
SLL Delete End	Walk till TEMP.next.next == NULL, cut TEMP.next
SLL Delete Mid	Stop at pos-1, DEL = TEMP.next, bridge over DEL

SESSION 2 OF 7

| Topics: Doubly Linked List + Stack

► TOPIC 3: DOUBLY LINKED LIST (DLL)

◆ THEORY

A **Doubly Linked List (DLL)** is a linear dynamic data structure where each node has **three parts**:

1. **PREV pointer** — address of the previous node
2. **DATA** — the actual value
3. **NEXT pointer** — address of the next node

```
NULL ← [PREV | 10 | NEXT] ↔ [PREV | 20 | NEXT] ↔ [PREV | 30 | NEXT] → NULL
                        HEAD
```

Key Rules: First node's PREV = NULL — Last node's NEXT = NULL — Can be traversed in **both directions** — HEAD points to first node.

Why DLL over SLL? Bidirectional traversal; Deletion is easier (no need to track previous node separately); More flexible but uses **extra memory** (one extra pointer per node).

◆ INSERTION ALGORITHMS

1. Insertion at Beginning

```
Algorithm: INSERT_BEGIN
Input: Element X
Output: Updated List

Steps:
1. Start
2. Create new node N, set N.data = X
3. Set N.prev = NULL
4. Set N.next = HEAD
5. If HEAD != NULL
   → Set HEAD.prev = N   ← old HEAD links back to N
6. Set HEAD = N
7. Stop
```

Complexity: O(1)

💡 Trick: "SLL insert_begin + one extra step: tell old HEAD to look back at N."

⚠ Extra step vs SLL: You must update **HEAD.prev = N** — this is what makes it doubly linked. Forgetting this = losing backward link.

2. Insertion at End

```
Algorithm: INSERT_END
Input: Element X
Output: Updated List

Steps:
1. Start
2. Create new node N, set N.data = X
3. If HEAD == NULL
   → Set HEAD = N, Stop
4. Set TEMP = HEAD
5. While TEMP.next != NULL
   → TEMP = TEMP.next
6. Set TEMP.next = N
7. Set N.prev = TEMP      ← N looks back at TEMP
8. Set N.next = NULL
9. Stop
```

Complexity: $O(n)$

💡 Trick: "Same as SLL insert_end + N.prev = TEMP. Walk, attach, look back."

⚠ Extra step vs SLL: After linking TEMP.next = N, also set **N.prev = TEMP**.

3. Insertion at Position (Middle)

```
Algorithm: INSERT_POS
Input: Element X, Position pos
Output: Updated List

Steps:
1. Start
2. Create new node N, set N.data = X
3. If pos == 0
   → Insert at beginning
4. Set TEMP = HEAD
5. Move TEMP to node at (pos-1)
6. Set N.next = TEMP.next
7. Set N.prev = TEMP
8. If TEMP.next != NULL
   → Set TEMP.next.prev = N   ← next node looks back at N
9. Set TEMP.next = N
10. Stop
```

Complexity: $O(n)$

💡 Trick: "4 pointer updates: N.next, N.prev, next_node.prev, TEMP.next — in this order."

⚠ Order is critical:

- Step 6 before Step 9 (set N.next before breaking TEMP.next link)
- Step 8 before Step 9 (update next node's prev before unlinking)

◆ DELETION ALGORITHMS

1. Deletion from Beginning

```
Algorithm: DELETE_BEGIN
Output: Updated List

Steps:
1. Start
2. If HEAD == NULL → Print "Underflow", Stop
3. Set TEMP = HEAD
4. Set HEAD = HEAD.next
5. If HEAD != NULL
   → Set HEAD.prev = NULL   ← new HEAD has no previous
6. Delete TEMP
7. Stop
```

Complexity: $O(1)$

💡 Trick: "Move HEAD forward. New HEAD looks back at nobody (prev = NULL)."

⚠ Extra step vs SLL: After HEAD = HEAD.next, set **HEAD.prev = NULL**.

2. Deletion from End

```
Algorithm: DELETE_END
Output: Updated List

Steps:
1. Start
2. If HEAD == NULL → Print "Underflow", Stop
3. If HEAD.next == NULL
   → Set HEAD = NULL, Stop
4. Set TEMP = HEAD
5. While TEMP.next != NULL
   → TEMP = TEMP.next       ← reach LAST node
6. Set TEMP.prev.next = NULL ← second-to-last cuts forward link
7. Delete TEMP
8. Stop
```

Complexity: $O(n)$

💡 Trick: "In DLL, walk to the LAST node (not second-to-last). Use TEMP.prev to reach second-to-last, then cut. Easier than SLL!"

⚠ Key Difference from SLL Delete End:

- SLL: stop at second-to-last (TEMP.next.next == NULL)
- DLL: stop at LAST node (TEMP.next == NULL), then use TEMP.prev to go back

3. Deletion at Position

```
Algorithm: DELETE_POS
```

Input: Position pos
Output: Updated List

Steps:

1. Start
2. If HEAD == NULL → Print "Underflow", Stop
3. If pos == 0
→ Delete from beginning
4. Set TEMP = HEAD
5. Move TEMP to node at position pos
6. Set TEMP.prev.next = TEMP.next
7. If TEMP.next != NULL
→ Set TEMP.next.prev = TEMP.prev
8. Delete TEMP
9. Stop

Complexity: $O(n)$

💡 Trick: "In DLL deletion at pos, move TO pos (not pos-1 like SLL). Use TEMP.prev and TEMP.next to stitch neighbors together."

⚠️ Key Difference from SLL Delete Pos:

- SLL: stop at **pos-1**, use a separate DEL pointer
- DLL: stop at **pos itself**, use TEMP.prev directly

◆ SEARCHING ALGORITHM

Algorithm: SEARCH
Input: Element X
Output: Position or "Not Found"

Steps:

1. Start
2. Set TEMP = HEAD, pos = 0
3. While TEMP != NULL
If TEMP.data == X
→ Return pos
TEMP = TEMP.next
pos = pos + 1
4. Return "Not Found"
5. Stop

Complexity: $O(n)$ — Identical logic to SLL search

◆ COMPLEXITY TABLE — DLL

Operation	Complexity
Insertion at Beginning	$O(1)$
Insertion at End	$O(n)$
Insertion at Position	$O(n)$
Deletion from Beginning	$O(1)$

Deletion from End	O(n)
Deletion at Position	O(n)
Searching	O(n)

◆ EXAM QUESTIONS — DOUBLY LINKED LIST

Q1. What is a Doubly Linked List? Draw its node structure.

A Doubly Linked List is a dynamic linear data structure in which each node contains three fields: a **PREV pointer** (pointing to the previous node), **DATA** (the stored value), and a **NEXT pointer** (pointing to the next node).

Node structure: [PREV | DATA | NEXT]

The first node's PREV = NULL and the last node's NEXT = NULL. The HEAD pointer identifies the start of the list. Unlike a singly linked list, DLL supports **bidirectional traversal**.

Q2. Dry Run: Insert 5 at beginning of 10↔20↔30.

- Create N, N.data = 5
- N.prev = NULL
- N.next = HEAD (HEAD points to 10) → N.next = 10
- HEAD != NULL → HEAD.prev = N (node 10's prev = N)
- HEAD = N
- **Result: 5↔10↔20↔30 ✓**

Q3. Dry Run: Delete from End in list 10↔20↔30↔40.

- HEAD != NULL ✓ — HEAD.next != NULL (not single node) ✓
- TEMP = HEAD = 10 → TEMP.next=20 (not NULL) → TEMP=20 → TEMP.next=30 (not NULL) → TEMP=30 → TEMP.next=40 (not NULL) → TEMP=40
- TEMP.next = NULL → Exit loop (TEMP is now at node 40)
- TEMP.prev.next = NULL → node 30's next = NULL
- Delete TEMP (node 40)
- **Result: 10↔20↔30 ✓**

Q4. What are the differences between SLL and DLL?

Feature	Singly Linked List	Doubly Linked List
Pointers per node	1 (next only)	2 (prev + next)
Traversal	Forward only	Both directions

Memory per node	Less	More
Deletion at end	Needs second-to-last (TEMP.next.next)	Needs last node (TEMP.next), use TEMP.prev
Delete at position	Stop at pos-1, use DEL pointer	Stop at pos, use TEMP.prev directly
Complexity	Same	Same
Implementation	Simpler	More complex

Q5. Why is DLL deletion from end easier than SLL deletion from end?

In SLL, to delete the last node, we must stop at the **second-to-last node** (using TEMP.next.next == NULL) because we have no way to go backward — we need to set the previous node's next = NULL manually.

In DLL, we simply walk to the **last node** (TEMP.next == NULL). Since every node has a PREV pointer, we use **TEMP.prev.next = NULL** to disconnect the last node without needing a second pointer or stopping early. This makes DLL deletion at end conceptually cleaner.

Q6. What is the difference between Insertion at Position in SLL vs DLL?

Step	SLL	DLL
Stop at	pos-1	pos-1
Pointers updated	N.next, TEMP.next	N.next, N.prev, TEMP.next.prev, TEMP.next
Backward link	Not needed	Must update TEMP.next.prev = N
Risk	Losing forward link if order wrong	Losing both links if order wrong

► TOPIC 4: STACK

◆ THEORY

A **Stack** is a linear data structure that follows the **LIFO** principle:

LIFO = Last In, First Out — The last element inserted is the first one removed.

Think of it as a **stack of plates** — you add and remove only from the top.

Implementation using Array: Array: STACK[MAX] — Variable: **TOP = -1** (means stack is empty) — TOP always points to the **most recently inserted element**.

Basic Operations: PUSH (Insert), POP (Remove), PEEK/TOP (View top), IsFull (overflow check), IsEmpty (underflow check).

Real-world applications: Undo/Redo in text editors — Browser back button — Function call management (recursion) — Expression evaluation (infix → postfix) — Syntax checking (balanced parentheses)

◆ ALGORITHMS

PUSH Operation

```
Algorithm: PUSH
Input: Element X
Output: Updated Stack

Steps:
1. Start
2. If TOP == MAX - 1
   → Print "Stack Overflow" and Exit
3. Else
   → TOP = TOP + 1
   → STACK[TOP] = X
4. Stop
```

Complexity: $O(1)$

💡 Trick: "PUSH = Check ceiling (TOP == MAX-1). If room, raise TOP, place X."

⚠ Condition: TOP == MAX-1 (not MAX) because index starts at 0. If MAX=5, valid indices are 0-4. When TOP=4, stack is full.

POP Operation

```
Algorithm: POP
Output: Deleted element X

Steps:
1. Start
2. If TOP == -1
   → Print "Stack Underflow" and Exit
3. Else
   → X = STACK[TOP]
   → TOP = TOP - 1
   → Return X
4. Stop
```

Complexity: $O(1)$

💡 Trick: "POP = Check floor (TOP == -1). If not empty, grab top element, lower TOP, return it."

⚠ Order in POP: Save element FIRST (X = STACK[TOP]), THEN decrease TOP. Reverse = data lost!

◆ COMPLEXITY TABLE — STACK

Operation	Complexity	Reason
PUSH	$O(1)$	Only updates TOP, no loop
POP	$O(1)$	Only updates TOP, no loop

PEEK	O(1)	Direct access via TOP
------	------	-----------------------

🔑 Both push and pop are O(1) — this is a major advantage of stacks.

◆ DRY RUN EXAMPLE

Stack size MAX = 5, initially TOP = -1

Operation	Action	TOP	Stack State
Push 10	STACK[0]=10	0	[10]
Push 20	STACK[1]=20	1	[10,20]
Push 30	STACK[2]=30	2	[10,20,30]
Pop	Returns 30, TOP--	1	[10,20]

◆ OVERFLOW & UNDERFLOW

Stack Overflow: Occurs when you try to PUSH into a **full stack**. Condition: **TOP == MAX - 1**

Stack Underflow: Occurs when you try to POP from an **empty stack**. Condition: **TOP == -1**

◆ EXAM QUESTIONS — STACK

Q1. What is a Stack? Explain LIFO with a real-life example.

A Stack is a linear data structure based on the **LIFO (Last In, First Out)** principle, meaning the last element inserted is the first one to be removed. All insertions and deletions occur at one end called the **TOP**.

Real-life example: A stack of plates in a cafeteria. You place (push) plates on top and always take (pop) from the top. The last plate placed is the first one picked up.

Implementation: Uses an array STACK[MAX] with a variable TOP initialized to -1, indicating an empty stack.

Q2. Write and explain the PUSH and POP algorithms. Show a dry run: Push 5, 10, 15 then Pop twice. MAX=5.

(Write both algorithms)

Dry Run:

- Initially: TOP = -1
- Push 5: TOP≠4, TOP=0, STACK[0]=5
- Push 10: TOP≠4, TOP=1, STACK[1]=10
- Push 15: TOP≠4, TOP=2, STACK[2]=15
- Stack: [5, 10, 15], TOP=2
- Pop: TOP≠-1, X=STACK[2]=15, TOP=1 → Returns **15**
- Pop: TOP≠-1, X=STACK[1]=10, TOP=0 → Returns **10**
- Stack: [5], TOP=0

Q3. What is Stack Overflow and Stack Underflow? When do they occur?

Stack Overflow occurs when an attempt is made to PUSH an element into a stack that is already full. This is detected by the condition: **TOP == MAX - 1**. Inserting beyond this limit causes overflow because no more array space is available.

Stack Underflow occurs when an attempt is made to POP an element from an empty stack. This is detected by the condition: **TOP == -1**. Removing from an empty stack has nothing to return, causing underflow.

Both are error conditions that must be checked before performing any stack operation.

Q4. What are the applications of Stack?

1. **Undo/Redo operations** — Each action is pushed; undo pops the last action
2. **Browser Back button** — Pages are pushed as visited; back pops the last page
3. **Function call stack** — Recursive function calls are stored in a stack
4. **Expression evaluation** — Converting infix to postfix/prefix uses a stack
5. **Balanced parentheses checking** — Opening brackets pushed, closing brackets pop and match
6. **Syntax parsing** in compilers

Q5. Difference between Stack and Queue.

Feature	Stack	Queue
Principle	LIFO	FIFO
Insertion end	TOP	REAR
Deletion end	TOP	FRONT
Operations	Push, Pop	Enqueue, Dequeue
Empty condition	TOP == -1	FRONT == -1
Full condition	TOP == MAX-1	REAR == MAX-1
Example	Stack of plates	Line at ticket counter

Q6. Why are Push and Pop O(1)?

Both Push and Pop operate only on the **TOP** of the stack. They never loop through elements — they simply increment or decrement the TOP variable and directly access `STACK[TOP]` using the array index. Since the number of operations does not depend on the size of the stack, both are constant time **O(1)** operations.

Q7. What happens if you Push when TOP == MAX-1? What is printed and why?

When $TOP == MAX-1$, the stack has reached its maximum capacity (all array positions 0 to $MAX-1$ are filled). Attempting another PUSH triggers the **overflow condition**. The algorithm prints "**Stack Overflow**" and exits without inserting the element, because there is no available memory location in the array to store a new value.

📌 EXAM STRATEGY — STACK

1. Always start theory with "**LIFO**" definition — it's the foundation
2. Mention **$TOP = -1$ means empty stack** — examiners check this
3. In POP algorithm — always note "element is saved BEFORE TOP is decremented"
4. **Overflow vs Underflow** is a high-probability question — prepare full answer
5. Applications question is frequently asked — memorize at least 4 points
6. Complexity must always include the **reason** (no loop = $O(1)$)

🔑 MASTER MEMORY TRICKS — SESSION 2

Algorithm	Core Trick to Remember
DLL Insert Begin	SLL insert begin + $HEAD.prev = N$
DLL Insert End	SLL insert end + $N.prev = TEMP$
DLL Insert Mid	4 pointers: $N.next \rightarrow N.prev \rightarrow next.prev \rightarrow TEMP.next$
DLL Delete Begin	Move $HEAD + HEAD.prev = NULL$
DLL Delete End	Walk to LAST node, use $TEMP.prev.next = NULL$
DLL Delete Pos	Walk to pos (not pos-1), use $TEMP.prev$ & $TEMP.next$ to bridge
PUSH	Check $TOP == MAX-1$, then $TOP++$, $STACK[TOP] = X$
POP	Check $TOP == -1$, then save $X = STACK[TOP]$, then $TOP--$

SESSION 3 OF 7

| Topics: Queue + Trees & BST

► TOPIC 5: QUEUE

◆ THEORY

A **Queue** is a linear data structure that follows the **FIFO** principle:

FIFO = First In, First Out — The first element inserted is the first one removed.

Think of it as a **line at a ticket counter** — first person in line is served first.

Implementation using Array: Array: `QUEUE[MAX]` — **FRONT** = **-1** (tracks removal end) — **REAR** = **-1** (tracks insertion end) — Both initialized to **-1** → means queue is **empty**.

Real-world applications: CPU scheduling — Printer spooling — BFS graph traversal — Call center waiting lines — Keyboard buffer

◆ ALGORITHMS

ENQUEUE Operation (Insert)

```
Algorithm: ENQUEUE
Input: Element X
Output: Updated Queue

Steps:
1. Start
2. If REAR == MAX - 1
   → Print "Queue Overflow" and Exit
3. If FRONT == -1
   → Set FRONT = 0           ← first element case
4. REAR = REAR + 1
5. QUEUE[REAR] = X
6. Stop
```

Complexity: $O(1)$

💡 Trick: "ENQUEUE enters from the REAR. Check ceiling ($REAR == MAX-1$). If first element, wake up FRONT. Then raise REAR and place X."

⚠ Critical conditions:

- Full: $REAR == MAX-1$
- First element ever: $FRONT == -1 \rightarrow$ set $FRONT = 0$
- FRONT is only set ONCE here (initialization) — never changed in ENQUEUE

DEQUEUE Operation (Delete)

```
Algorithm: DEQUEUE
Output: Deleted element X

Steps:
1. Start
2. If FRONT == -1 OR FRONT > REAR
   → Print "Queue Underflow" and Exit
3. X = QUEUE[FRONT]
4. If FRONT == REAR
   → Set FRONT = REAR = -1 ← last element removed
   Else
   → FRONT = FRONT + 1
5. Return X
6. Stop
```

Complexity: $O(1)$

💡 Trick: "DEQUEUE exits from the FRONT. Save element, move FRONT forward. If it was the LAST element (FRONT==REAR), reset both to -1."

⚠ Two empty conditions: FRONT == -1 (never inserted) OR FRONT > REAR (all removed)

⚠ Reset condition: When FRONT == REAR before removal (one element), reset FRONT = REAR = -1 after removal

ISFULL Operation

```
Algorithm: ISFULL
Steps:
1. Start
2. If REAR == MAX - 1 → Return TRUE
3. Else → Return FALSE
4. Stop
```

ISEMPTY Operation

```
Algorithm: ISEMPTY
Steps:
1. Start
2. If FRONT == -1 OR FRONT > REAR → Return TRUE
3. Else → Return FALSE
4. Stop
```

💡 Trick: ISEMPTY condition is **identical** to DEQUEUE underflow condition — same logic, different purpose.

◆ COMPLEXITY TABLE — QUEUE

Operation	Complexity	Reason
ENQUEUE	$O(1)$	Only updates REAR
DEQUEUE	$O(1)$	Only updates FRONT

ISFULL	O(1)	Single comparison
ISEMPTY	O(1)	Single comparison

◆ DRY RUN (From PDF)

Queue MAX=5, FRONT=-1, REAR=-1

Operation	Action	FRONT	REAR	Queue
Enqueue 10	FRONT=0, REAR=0, Q[0]=10	0	0	[10]
Enqueue 20	REAR=1, Q[1]=20	0	1	[10,20]
Enqueue 30	REAR=2, Q[2]=30	0	2	[10,20,30]
Dequeue	X=Q[0]=10, FRONT=1	1	2	[20,30]

◆ LIMITATION OF LINEAR QUEUE

Even after dequeuing, front positions are **wasted** — once REAR hits MAX-1, no more inserts even if front slots are empty. This is solved by a **Circular Queue**.

◆ EXAM QUESTIONS — QUEUE

Q1. What is a Queue? Explain FIFO with a real-life example.

A Queue is a linear data structure based on the **FIFO (First In, First Out)** principle — the first element inserted is the first to be removed. It has two ends: **FRONT** (for deletion) and **REAR** (for insertion).

Real-life example: A line of people at a bus stop. The first person to arrive (first in) is the first to board the bus (first out). No one can cut in from the middle.

Implementation: Uses array QUEUE[MAX] with FRONT = -1 and REAR = -1 initially, both indicating an empty queue.

Q2. Dry run: Enqueue 5, 10, 15 then Dequeue twice. MAX=5.

- Initially: FRONT=-1, REAR=-1
- Enqueue 5: REAR=-1 $\neq$ 4 ✓, FRONT=-1 $\rightarrow$ set FRONT=0, REAR=0, Q[0]=5
- Enqueue 10: REAR=0 $\neq$ 4 ✓, FRONT $\neq$ -1, REAR=1, Q[1]=10
- Enqueue 15: REAR=1 $\neq$ 4 ✓, REAR=2, Q[2]=15
- State: FRONT=0, REAR=2, Queue=[5,10,15]
- Dequeue: FRONT=0, not > REAR=2 ✓, X=Q[0]=5, FRONT $\neq$ REAR $\rightarrow$ FRONT=1 $\rightarrow$ **Returns 5**
- Dequeue: FRONT=1 $\neq$ REAR=2 ✓, X=Q[1]=10, FRONT=2 $\rightarrow$ **Returns 10**
- Final: FRONT=2, REAR=2, Queue=[15]

Q3. What is Queue Overflow and Queue Underflow?

Queue Overflow occurs when ENQUEUE is attempted on a full queue. Condition: **REAR == MAX-1**. No

more space at the rear.

Queue Underflow occurs when DEQUEUE is attempted on an empty queue. Conditions: **FRONT == -1** OR **FRONT > REAR**. Nothing to remove.

Q4. Why do we reset **FRONT = REAR = -1** in DEQUEUE?

When **FRONT == REAR** before a dequeue, it means only **one element remains** in the queue. After removing it, resetting both to -1 ensures the queue returns to its **original empty state**, allowing future enqueue operations to reinitialize **FRONT** properly (via the **FRONT == -1** check in **ENQUEUE**). Without this reset, re-inserting after full removal may malfunction.

Q5. What is a Circular Queue and why is it needed?

A **Circular Queue** is a queue where the last position connects back to the first, forming a circle. It is needed because in a **linear queue**, once **REAR** reaches **MAX-1**, no more insertions are possible — even if front positions are vacant due to dequeuing. This wastes memory.

In a circular queue, **REAR** wraps around using modulo: **REAR = (REAR + 1) % MAX**, reusing freed front positions. This eliminates the memory wastage problem of linear queues.

► TOPIC 6: TREES

◆ THEORY

A **Tree** is a **non-linear, hierarchical** data structure consisting of nodes connected by edges.

```
      10          ← Root
     /  \
    20   30      ← Internal Nodes
   /  \  /  \
  40  50 60      ← Leaf Nodes
```

Key Terminology (MUST MEMORIZE)

Term	Definition
Root	Topmost node, no parent
Node	Each element in the tree
Edge	Connection between two nodes
Parent	Node with children
Child	Node below a parent
Leaf Node	Node with NO children
Internal Node	Node with at least one child
Subtree	Any node + all its descendants

Height of Tree	Number of edges on longest path from root to leaf
Depth of Node	Number of edges from root to that node
Degree of Node	Number of children a node has
Level	Root is at Level 0 (or 1, depends on convention)

◆ BINARY TREE

A **Binary Tree** is a tree where each node has **at most 2 children**: Left child and Right child.

Type	Definition
Full Binary Tree	Every node has 0 or 2 children (no node has 1 child)
Complete Binary Tree	All levels filled except possibly last; last level filled left to right
Perfect Binary Tree	All internal nodes have 2 children + all leaves at same level
Skewed Tree	All nodes lean one side (left-skewed or right-skewed)

► TOPIC 7: BINARY SEARCH TREE (BST)

◆ THEORY

BST Property: For every node N:

- All values in **Left subtree** < **N.data**
- All values in **Right subtree** > **N.data**

```

      50
     /  \
    /    \
   /      \
  20      30  70
 / \  /
20 40 60 80

```

Why BST? Searching is faster: $O(\log n)$ average — Maintains sorted order — Efficient insertion and deletion.

◆ BST OPERATIONS

1. BST INSERTION

```

Algorithm: BST_INSERT
Input: Root, Element X
Output: Updated BST

Steps:
1. Start
2. Create new node N, set N.data = X, N.left = N.right = NULL
3. If Root == NULL
   → Set Root = N, Stop      ← empty tree
4. Set TEMP = Root
5. Loop:
   a. If X < TEMP.data
      → If TEMP.left == NULL
         → Set TEMP.left = N, Stop

```

```

        → Else TEMP = TEMP.left
    b. If X > TEMP.data
        → If TEMP.right == NULL
            → Set TEMP.right = N, Stop
        → Else TEMP = TEMP.right
    c. If X == TEMP.data
        → Print "Duplicate, not inserted", Stop
6. Stop

```

Complexity: $O(\log n)$ average, $O(n)$ worst (skewed tree)

💡 Trick: "Go LEFT if smaller, RIGHT if bigger. When you find a NULL spot, plant the node there."

2. BST SEARCHING

```

Algorithm: BST_SEARCH
Input: Root, Element X
Output: Found / Not Found

Steps:
1. Start
2. If Root == NULL
    → Return "Not Found"
3. Set TEMP = Root
4. While TEMP != NULL
    a. If X == TEMP.data → Return "Found"
    b. If X < TEMP.data → TEMP = TEMP.left
    c. If X > TEMP.data → TEMP = TEMP.right
5. Return "Not Found"
6. Stop

```

Complexity: $O(\log n)$ average, $O(n)$ worst

3. BST DELETION — THREE CASES

```

Algorithm: BST_DELETE
Input: Root, Element X

Steps:
1. Start
2. Search for node containing X
3. If not found → Print "Not Found", Stop
4. If node is leaf (no children)
    → Delete node, set parent pointer = NULL
5. If node has one child
    → Replace node with its child
6. If node has two children
    → Find inorder successor (leftmost node in right subtree)
    → Copy inorder successor's data into current node
    → Delete the inorder successor
7. Stop

```

💡 Trick: "0 children = just delete. 1 child = bypass. 2 children = swap with inorder successor then delete successor."

◆ TREE TRAVERSALS

INORDER (Left → Root → Right)

```
Algorithm: INORDER(node)
1. If node == NULL → Return
2. INORDER(node.left)
3. Visit node.data
4. INORDER(node.right)
```

💡 Result: Gives nodes in **sorted/ascending order** for BST!

PREORDER (Root → Left → Right)

```
Algorithm: PREORDER(node)
1. If node == NULL → Return
2. Visit node.data
3. PREORDER(node.left)
4. PREORDER(node.right)
```

💡 Use: Used to **copy/recreate** a tree (root first, then children)

POSTORDER (Left → Right → Root)

```
Algorithm: POSTORDER(node)
1. If node == NULL → Return
2. POSTORDER(node.left)
3. POSTORDER(node.right)
4. Visit node.data
```

💡 Use: Used to **delete** a tree (children first, then root)

◆ TRAVERSAL DRY RUN

```
      50
     /  \
    /    \
   /      \
  20      30  70
   \      /  \
    40    /  \
```

Inorder (L→Root→R): 20, 30, 40, 50, 70

Preorder (Root→L→R): 50, 30, 20, 40, 70

Postorder (L→R→Root): 20, 40, 30, 70, 50

💡 Quick Memory: "In-Pre-Post tells you WHERE the root is visited: IN=middle, PRE=first, POST=last"

◆ BST INSERTION DRY RUN

Insert: 50, 30, 70, 20, 40 into empty BST

- Insert 50 → Root = 50
- Insert 30 → 30 < 50 → go left → left empty → 30 becomes left child of 50
- Insert 70 → 70 > 50 → go right → right empty → 70 becomes right child of 50

- Insert 20 → 20 < 50 → left → 20 < 30 → left → empty → 20 = left child of 30
- Insert 40 → 40 < 50 → left → 40 > 30 → right → empty → 40 = right child of 30

```

      50
     /
    /
   /
  20  30  70
     /
    40

```

◆ EXAM QUESTIONS — TREES & BST

Q1. What is a Tree? Define all key terminologies.

A Tree is a **non-linear hierarchical data structure** consisting of nodes connected by edges. Unlike arrays and linked lists, a tree has a parent-child relationship among nodes. **Key rule:** A tree with N nodes has exactly **N-1 edges**.

(Draw tree diagram and define all terms from terminology table above)

Q2. Insert into empty BST and draw the tree: 40, 20, 60, 10, 30, 50, 70

- 40 → Root; 20 < 40 → Left of 40; 60 > 40 → Right of 40
- 10 < 40 → left → 10 < 20 → Left of 20; 30 < 40 → left → 30 > 20 → Right of 20
- 50 > 40 → right → 50 < 60 → Left of 60; 70 > 40 → right → 70 > 60 → Right of 60

```

      40
     /
    /
   / \
  10  20  60
     / \
    30 50 70

```

This is a **Perfect Binary Tree** — all levels completely filled.

Inorder: 10, 20, 30, 40, 50, 60, 70 ← SORTED ✓

Preorder: 40, 20, 10, 30, 60, 50, 70

Postorder: 10, 30, 20, 50, 70, 60, 40

Q3. Explain the three cases of BST Deletion with examples.

Case 1 — Leaf Node (no children):

Delete node 10. Node 10 has no children. Simply remove it and set its parent (20)'s left pointer to NULL.

Case 2 — One Child:

Delete node 60 if it only had child 70. Replace node 60 with its only child 70. Parent (40)'s right now points to 70.

Case 3 — Two Children:

Delete node 20 which has children 10 and 30. Find **inorder successor** = smallest in right subtree of 20 = **30**. Copy 30's data into node 20's position. Delete original node 30 (which is now a leaf).

Q4. Difference between Binary Tree and Binary Search Tree.

Feature	Binary Tree	Binary Search Tree
Ordering	No specific order	Left < Root < Right
Search	$O(n)$ — must check all	$O(\log n)$ average
Inorder result	Random order	Always sorted (ascending)
Duplicate handling	Allowed	Usually not allowed
Purpose	General hierarchy	Efficient search/sort

Q5. What is the time complexity of BST operations and when does worst case occur?

Operation	Average Case	Worst Case
Search	$O(\log n)$	$O(n)$
Insertion	$O(\log n)$	$O(n)$
Deletion	$O(\log n)$	$O(n)$

Worst case occurs when the BST becomes **skewed** — elements inserted in sorted order (e.g., 10, 20, 30, 40), making the tree look like a linked list. Every operation must traverse all n nodes $\rightarrow O(n)$.

🔑 MASTER MEMORY TRICKS — SESSION 3

Topic	Core Trick
ENQUEUE	Check $REAR == MAX-1$, first element: set $FRONT=0$, then $REAR++$ and insert
DEQUEUE	Check empty, save $X=Q[FRONT]$, if last element reset both to -1 else $FRONT++$
BST Insert	Left if smaller, Right if bigger, plant at NULL spot
BST Search	Same left/right logic, return found/not found
BST Delete	0 child=delete, 1 child=bypass, 2 children=inorder successor swap
Inorder	$L \rightarrow Root \rightarrow R \rightarrow$ gives SORTED order in BST
Preorder	$Root \rightarrow L \rightarrow R \rightarrow$ Root visited FIRST
Postorder	$L \rightarrow R \rightarrow Root \rightarrow$ Root visited LAST

📁 SESSION 4 OF 7

| Topics: Tree Traversal (Full) + Graphs + DFS + BFS

▶ TOPIC 8: TREE TRAVERSAL (COMPLETE & DETAILED)

◆ WHAT IS TREE TRAVERSAL?

Tree Traversal means visiting **every node** in a tree **exactly once** in a specific, systematic order. Unlike arrays or linked lists where traversal is always left-to-right, trees are **non-linear** — so we need defined strategies to visit all nodes.

Why traversal matters: To search for an element — To display all elements — To copy or delete a tree — To evaluate expressions stored in trees — To sort data (BST Inorder)

Three classical traversal methods — all **recursive**, all based on **when the ROOT is visited**.

◆ TRAVERSAL TYPE 1: INORDER (L → Root → R)

In Inorder traversal, we visit nodes in this exact sequence: (1) First — go to **Left subtree** recursively; (2) Second — visit **Root** (print/process it); (3) Third — go to **Right subtree** recursively.

Most important property: For a BST, Inorder traversal **always produces output in ascending (sorted) order**.

```
Algorithm: INORDER
Input: Root node of tree
Output: Nodes printed in Left→Root→Right order

Steps:
1. Start
2. If Root == NULL
   → Return          ← base case, stop recursion
3. INORDER(Root.left) ← Step 1: traverse left subtree
4. Print Root.data    ← Step 2: visit root
5. INORDER(Root.right) ← Step 3: traverse right subtree
6. Stop
```

Complexity: $O(n)$ — every node visited exactly once

💡 Trick: "**IN** = root in the **IN**side/middle. Left, then **YOU** (root), then Right."

◆ TRAVERSAL TYPE 2: PREORDER (Root → L → R)

In Preorder traversal: (1) First — visit **Root**; (2) Second — go to **Left subtree**; (3) Third — go to **Right subtree**.

Key use: Preorder is used to **copy a tree** or **serialize** it.

```
Algorithm: PREORDER
```



```
Input: Root node of tree
Output: Nodes printed in Root→Left→Right order
```

Steps:

1. Start
2. If `Root == NULL`
 - Return ← base case
3. Print `Root.data` ← Step 1: visit root FIRST
4. `PREORDER(Root.left)` ← Step 2: traverse left subtree
5. `PREORDER(Root.right)` ← Step 3: traverse right subtree
6. Stop

Complexity: $O(n)$

💡 Trick: "**PRE** = ROOT comes **PRE**viously (before everything). Root first, then explore."

◆ TRAVERSAL TYPE 3: POSTORDER ($L \rightarrow R \rightarrow \text{Root}$)

In Postorder traversal: (1) First — go to **Left subtree**; (2) Second — go to **Right subtree**; (3) Third — visit **Root**.

Key use: Postorder is used to **delete a tree** (children must be deleted before parent) and to **evaluate arithmetic expression trees**.

```
Algorithm: POSTORDER
Input: Root node of tree
Output: Nodes printed in Left→Right→Root order

Steps:
1. Start
2. If Root == NULL
   → Return ← base case
3. POSTORDER(Root.left) ← Step 1: traverse left subtree
4. POSTORDER(Root.right) ← Step 2: traverse right subtree
5. Print Root.data ← Step 3: visit root LAST
6. Stop
```

Complexity: $O(n)$

💡 Trick: "**POST** = ROOT comes **POST**eriorly (after everything). Children first, root last."

◆ COMPLETE TRAVERSAL DRY RUN — STEP BY STEP

```
      50
     /  \
    /    \
   /      \
  30      70
 /  \    /  \
20  40 60  80
```

INORDER ($L \rightarrow \text{Root} \rightarrow R$) — Recursive trace:

- Call `INORDER(50)` → Call `INORDER(30)` → Call `INORDER(20)` → `INORDER(NULL)`=return → **Print 20** → `INORDER(NULL)`=return
- Back to 30 → **Print 30** → Call `INORDER(40)` → **Print 40**
- Back to 50 → **Print 50** → Call `INORDER(70)` → **Print 60** → **Print 70** → **Print 80**

Inorder Result: 20, 30, 40, 50, 60, 70, 80 ← SORTED! ✓

PREORDER (Root→L→R):

Print 50 → go left → **Print 30** → go left → **Print 20** → both NULL → backtrack → **Print 40** → backtrack to 50 → go right → **Print 70** → go left → **Print 60** → go right → **Print 80**

Preorder Result: 50, 30, 20, 40, 70, 60, 80

POSTORDER (L→R→Root):

Go left all the way → **Print 20** → Backtrack → go right → **Print 40** → Both children done → **Print 30** → Go right subtree → **Print 60** → **Print 80** → **Print 70** → Both subtrees done → **Print 50**

Postorder Result: 20, 40, 30, 60, 80, 70, 50

◆ EXPRESSION TREE

```
      +
     / \
    /   * 5
   /  \
  3    4
```

- **Inorder:** 3 * 4 + 5 → **Infix expression**
- **Preorder:** + * 3 4 5 → **Prefix expression**
- **Postorder:** 3 4 * 5 + → **Postfix expression**

💡 Trick: "Inorder gives Infix. Pre gives Prefix. Post gives Postfix. **The name matches!**"

◆ TRAVERSAL COMPARISON TABLE

Feature	Inorder	Preorder	Postorder
Order	L → Root → R	Root → L → R	L → R → Root
Root position	Middle	First	Last
BST gives sorted?	✓ Yes (ascending)	✗ No	✗ No
Used for	Sorting, display	Copying tree	Deleting tree
Expression type	Infix	Prefix	Postfix
Complexity	O(n)	O(n)	O(n)

◆ EXAM QUESTIONS — TREE TRAVERSAL

Q1. What is tree traversal? Why is it needed? Name the three types.

Tree traversal is the process of **visiting every node in a tree exactly once** in a systematic, predefined order. Unlike linear data structures where traversal follows a single path, trees are non-linear and hierarchical, so explicit traversal strategies are required.

Why it is needed: To search for a specific value — To display all elements — To copy or delete the entire tree — To evaluate arithmetic expression trees — To sort data (Inorder of BST yields sorted output).

Three types: (1) **Inorder** — Left → Root → Right — (2) **Preorder** — Root → Left → Right — (3) **Postorder** — Left → Right → Root. All three have **O(n)** time complexity.

Q2. Perform all three traversals on this tree:

```
      40
     /  \
    /    \
   /      \
  10      20  60
   \      / \
   30     30
```

Inorder (L→Root→R):

Left of 40 → 20 → Left of 20 → 10 (leaf) → Print **10** → Back to 20 → Print **20** → Right → Print **30** → Back to 40 → Print **40** → Right → Print **60**

Result: 10, 20, 30, 40, 60 ← Sorted ascending ✓

Preorder (Root→L→R): Print **40** → go left → Print **20** → go left → Print **10** → backtrack → go right → Print **30** → back to 40 → go right → Print **60**

Result: 40, 20, 10, 30, 60

Postorder (L→R→Root): Go deepest left → Print **10** → Print **30** → Back to 20 → Print **20** → Right of 40 → Print **60** → Root → Print **40**

Result: 10, 30, 20, 60, 40

Q3. What is the significance of Inorder traversal in a BST?

The most important property of Inorder traversal on a Binary Search Tree is that it always produces the node values in **strictly ascending (sorted) order**. This happens because the BST property guarantees that for any node, all left subtree values are smaller and all right subtree values are larger. Inorder visits Left first, then Root, then Right — naturally producing sorted order at every level of recursion.

Practical significance: BST + Inorder traversal = **Tree Sort algorithm** — Used to verify whether a given binary tree is a valid BST — Used in database indexing for range queries.

Q4. What is an Expression Tree? Evaluate using traversals on: -(+, 6 2, 8)

```
      -
     /  \
    /    \
   /      \
  6      +  8
   \      / \
   2     6  2
```

An Expression Tree is a binary tree where **leaf nodes** contain **operands** (numbers) and **internal nodes** contain **operators** (+, -, *, /).

Inorder (L → Root → R) → Infix: 6 + 2 - 8

Preorder (Root → L → R) → Prefix: - + 6 2 8

Postorder (L → R → Root) → Postfix: 6 2 + 8 -

Evaluation: 6 + 2 = 8 → 8 - 8 = **0** ✓

Q5. Differentiate Preorder and Postorder. When is each used?

Feature	Preorder	Postorder
Visit order	Root $\rightarrow$ L $\rightarrow$ R	L $\rightarrow$ R $\rightarrow$ Root
Root visited	First	Last
Primary use	Copying/replicating a tree	Deleting a tree
Expression	Prefix notation	Postfix notation

Preorder use: When creating a copy, you must record the root before its children — inserting nodes in Preorder order recreates the original tree structure exactly.

Postorder use: When deleting a tree, children must be deleted before parents — Postorder ensures leaves are deleted first, root is deleted last safely.

Q6. Given Inorder: 4,2,5,1,6,3,7 and Preorder: 1,2,4,5,3,6,7 — reconstruct the tree.

Rule: In Preorder, the **first element is always the Root**.

- Preorder first = **1** $\rightarrow$ Root = 1
- Find 1 in Inorder $\rightarrow$ left of 1: {4,2,5} = Left subtree; {6,3,7} = Right subtree
- Next Preorder = **2** $\rightarrow$ root of left subtree; Find 2 in {4,2,5} $\rightarrow$ left: {4}, right: {5}
- Next relevant = **3** $\rightarrow$ root of right subtree {6,3,7}; left: {6}, right: {7}

```

      1
     / \
    2   3
   / \ / \
  4  5 6  7

```

► TOPIC 9: GRAPHS

◆ THEORY

A **Graph** is a **non-linear data structure** consisting of: A set of **Vertices (Nodes)** — represented as V; and A set of **Edges** — represented as E, connecting pairs of vertices.

Formal definition: $G = (V, E)$

```

1 — 2
| / |
| / |
| / |
3 — 4

```

Here: $V = \{1,2,3,4\}$, $E = \{(1,2),(1,3),(2,3),(2,4),(3,4)\}$

Key difference from Trees: Trees have root, parent-child hierarchy, N-1 edges, no cycles. Graphs have **no root**, no strict hierarchy, can have **cycles**, and any number of edges. A Tree is actually a **special case of a graph** — specifically a connected, acyclic, undirected graph.

◆ GRAPH TERMINOLOGY

Term	Definition
Vertex/Node	Basic unit of graph
Edge	Connection between two vertices
Adjacent vertices	Vertices connected by an edge
Degree of vertex	Number of edges connected to it
Path	Sequence of vertices connected by edges
Cycle	Path that starts and ends at same vertex
Connected Graph	Every vertex reachable from every other
Weighted Graph	Edges have weights/costs
Unweighted Graph	Edges have no weight, just connections

◆ TYPES OF GRAPHS

- 1. Directed Graph (Digraph):** Edges have direction (arrows). Edge $A \rightarrow B$ means you can go from A to B but NOT necessarily B to A.
- 2. Undirected Graph:** Edges have no direction. Edge (A,B) means you can travel both $A \rightarrow B$ and $B \rightarrow A$.
- 3. Weighted Graph:** Each edge carries a weight (cost/distance). Used in shortest path algorithms.
- 4. Complete Graph:** Every vertex is connected to every other vertex. n vertices = $n(n-1)/2$ edges.
- 5. Cyclic vs Acyclic:** Cyclic = contains at least one cycle. Acyclic = no cycles (trees are acyclic graphs).

◆ GRAPH REPRESENTATIONS

1. Adjacency Matrix — 2D array of size $V \times V$

Matrix[i][j] = 1 if edge exists between vertex i and j; = 0 if no edge.

For $V = \{1, 2, 3, 4\}$, $E = \{(1, 2), (1, 3), (2, 3), (2, 4), (3, 4)\}$:

```

      1  2  3  4
1 [ 0  1  1  0 ]
2 [ 1  0  1  1 ]
3 [ 1  1  0  1 ]
4 [ 0  1  1  0 ]

```

Advantages: Simple, $O(1)$ edge lookup — **Disadvantages:** $O(V^2)$ space, wasteful for sparse graphs

2. Adjacency List — Each vertex has a linked list of neighbors

```

1 → [2, 3]
2 → [1, 3, 4]
3 → [1, 2, 4]
4 → [2, 3]

```

Advantages: Space efficient $O(V+E)$ — **Disadvantages:** Edge lookup is $O(\text{degree of vertex})$

◆ ADJACENCY MATRIX vs ADJACENCY LIST

Feature	Adjacency Matrix	Adjacency List
Space	$O(V^2)$	$O(V + E)$

Edge check	O(1)	O(degree)
Add vertex	Expensive	Easy
Best for	Dense graphs	Sparse graphs
Implementation	2D array	Array of lists

◆ EXAM QUESTIONS — GRAPHS

Q1. What is a Graph? How is it different from a Tree?

A Graph $G = (V, E)$ is a non-linear data structure consisting of a finite set of **vertices (V)** and a set of **edges (E)** that connect pairs of vertices. Graphs model relationships, networks, maps, and connections between entities.

Feature	Tree	Graph
Structure	Hierarchical	Network (arbitrary)
Root	Has one root	No root
Cycles	No cycles	May have cycles
Edges	Exactly N-1 for N nodes	Any number
Type	Special case of graph	General structure

Q2. Draw adjacency matrix and list for: Vertices {A,B,C,D}, Edges {(A,B),(A,C),(B,D),(C,D)}

Graph:

```

A — B
|     |
C — D

```

Adjacency Matrix:

```

      A  B  C  D
A  [ 0  1  1  0 ]
B  [ 1  0  0  1 ]
C  [ 1  0  0  1 ]
D  [ 0  1  1  0 ]

```

Adjacency List:

```

A → [B, C]
B → [A, D]
C → [A, D]
D → [B, C]

```

◆ THEORY

DFS explores a graph by going as **deep as possible** along one path before backtracking and trying another path.

Analogy: Exploring a maze — you keep going forward into one path as deep as you can. When you hit a dead end, you backtrack and try another direction.

Data Structure Used: STACK (either explicitly or via recursion call stack)

◆ DFS ALGORITHM

```
Algorithm: DFS
Input: Graph G, Starting vertex S
Output: All vertices visited in DFS order

Steps:
1. Start
2. Create an empty STACK
3. Create a VISITED array, mark all vertices as NOT VISITED
4. Push starting vertex S onto STACK
5. While STACK is not empty:
    a. POP a vertex V from STACK
    b. If V is NOT VISITED:
        → Mark V as VISITED
        → Print V
        → For each neighbor W of V (in adjacency list):
            → If W is NOT VISITED:
                → Push W onto STACK
6. Stop
```

Complexity: $O(V + E)$ using Adjacency List — $O(V^2)$ using Adjacency Matrix

💡 Trick: "DFS = Go DEEP first. Use a STACK. Push neighbors, pop and visit. Backtrack when stuck."

⚠ Must maintain a **VISITED array** to avoid revisiting and infinite loops.

⚠ Mark as visited when POPPED (not when pushed in iterative DFS).

Recursive DFS

```
Algorithm: DFS_RECURSIVE
Input: Graph G, Vertex V, Visited[]

Steps:
1. Mark V as VISITED
2. Print V
3. For each neighbor W of V:
    If W is NOT VISITED:
        → Call DFS_RECURSIVE(G, W, Visited)
```

💡 Note: Recursive DFS is cleaner — the call stack automatically handles backtracking.

◆ DFS DRY RUN

Graph:

```
1 — 2 — 4
|       |
3 — 5
```

Adjacency List:

1: [2, 3], 2: [1, 4, 5], 3: [1, 5], 4: [2], 5: [2, 3]

Starting vertex = 1

Step	Action	Stack	Visited
1	Push 1	[1]	{}
2	Pop 1, mark visited, push neighbors 2,3	[2,3]	{1}
3	Pop 3, mark visited, push 5 (1 visited)	[2,5]	{1,3}
4	Pop 5, mark visited, push 2 (3 visited)	[2,2]	{1,3,5}
5	Pop 2, mark visited, push 4 (1,5 visited)	[2,4]	{1,3,5,2}
6	Pop 4, mark visited, no unvisited neighbors	[2]	{1,3,5,2,4}
7	Pop 2, already visited, skip	[]	All done

DFS Order: 1, 3, 5, 2, 4

► TOPIC 11: BFS (BREADTH FIRST SEARCH)

◆ THEORY

BFS explores a graph **level by level** — visits all neighbors of a vertex before going deeper.

Analogy: Ripples in a pond — when you drop a stone, ripples spread outward in rings. BFS visits the source, then all vertices 1 step away, then all 2 steps away, and so on.

Data Structure Used: QUEUE (FIFO — this ensures level-by-level exploration)

◆ BFS ALGORITHM

Algorithm: BFS

Input: Graph G, Starting vertex S

Output: All vertices visited in BFS order

Steps:

1. Start
2. Create an empty QUEUE
3. Create a VISITED array, mark all as NOT VISITED
4. Mark S as VISITED
5. ENQUEUE S into QUEUE
6. While QUEUE is not empty:
 - a. DEQUEUE vertex V from QUEUE
 - b. Print V
 - c. For each neighbor W of V:
 - If W is NOT VISITED:
 - Mark W as VISITED
 - ENQUEUE W

Complexity: $O(V + E)$ using Adjacency List — $O(V^2)$ using Adjacency Matrix

💡 Trick: "BFS = Go WIDE first. Use a QUEUE. Enqueue neighbors, dequeue and visit. Layer by layer."

⚠ Critical: Mark vertex as visited when **ENQUEUED** (not when dequeued) — prevents duplicates in queue.

◆ BFS DRY RUN

Same Graph as DFS, Starting vertex = 1

Step	Action	Queue	Visited
1	Enqueue 1, mark visited	[1]	{1}
2	Dequeue 1, print 1, enqueue 2,3 (mark visited)	[2,3]	{1,2,3}
3	Dequeue 2, print 2, enqueue 4,5 (1 visited)	[3,4,5]	{1,2,3,4,5}
4	Dequeue 3, print 3, neighbors all visited	[4,5]	{1,2,3,4,5}
5	Dequeue 4, print 4, 2 already visited	[5]	{1,2,3,4,5}
6	Dequeue 5, print 5, all neighbors visited	[]	All done

BFS Order: 1, 2, 3, 4, 5 ← Level by level ✓

◆ DFS vs BFS — MASTER COMPARISON TABLE

Feature	DFS	BFS
Full Form	Depth First Search	Breadth First Search
Strategy	Go as deep as possible	Go level by level
Data Structure	Stack (LIFO)	Queue (FIFO)
Implementation	Recursive or iterative	Iterative (usually)
Memory usage	Less (only current path)	More (entire level stored)
Finds shortest path?	✗ No	✓ Yes (unweighted)
Time Complexity	$O(V+E)$ adj list	$O(V+E)$ adj list
Visited marking	When POPPED	When ENQUEUED
Applications	Maze solving, cycle detection, topological sort	Shortest path, social networks, GPS

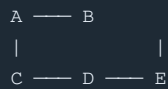
◆ EXAM QUESTIONS — DFS & BFS

Q1. What is DFS? Explain with algorithm and dry run. Vertices: {A,B,C,D,E}, Edges: {(A,B),(A,C),(B,D),(C,D),(D,E)}, Start: A

DFS (Depth First Search) is a graph traversal algorithm that explores as far as possible along each

branch before backtracking. It uses a **STACK** data structure (or recursion).

Graph:



Adjacency List: A: [B,C], B: [A,D], C: [A,D], D: [B,C,E], E: [D]

Step	Action	Stack	Visited
1	Push A	[A]	{}
2	Pop A, visited, push B,C	[B,C]	{A}
3	Pop C, visited, push D (A visited)	[B,D]	{A,C}
4	Pop D, visited, push B,E	[B,B,E]	{A,C,D}
5	Pop E, visited	[B,B]	{A,C,D,E}
6	Pop B, visited	[B]	{A,C,D,E,B}
7	Pop B, already visited	[]	All done

DFS Order: A, C, D, E, B

Q2. What is BFS? Dry run on same graph above. Start: A

BFS (Breadth First Search) is a graph traversal algorithm that explores all neighbors of a vertex before moving to the next level. Uses a **QUEUE**.

Step	Action	Queue	Visited
1	Enqueue A, mark visited	[A]	{A}
2	Dequeue A, print A, enqueue B,C	[B,C]	{A,B,C}
3	Dequeue B, print B, enqueue D	[C,D]	{A,B,C,D}
4	Dequeue C, print C, D already visited	[D]	{A,B,C,D}
5	Dequeue D, print D, enqueue E	[E]	{A,B,C,D,E}
6	Dequeue E, print E	[]	All done

BFS Order: A, B, C, D, E ← Level by level ✓

Q3. Why does DFS use a Stack and BFS use a Queue? Explain the logical reason.

DFS uses a Stack (LIFO): When we push a vertex's neighbors onto a stack and then pop, we always get the **most recently added** neighbor — this causes us to go deeper into the last discovered path before exploring alternatives. LIFO behavior = depth-first exploration.

BFS uses a Queue (FIFO): When we enqueue neighbors and then dequeue, we always process the **oldest added** vertex first — this means all vertices at distance 1 from source are processed before distance 2 vertices. FIFO behavior = breadth-first, level-by-level exploration.

In essence: Stack makes you dive deep. Queue makes you spread wide.

Q4. Which traversal finds the shortest path — DFS or BFS? Why?

BFS finds the shortest path in an **unweighted graph** because it visits vertices in order of their distance from the source — the first time BFS reaches a vertex, it has taken the minimum number of steps (edges). All vertices at distance k are visited before any vertex at distance $k+1$.

DFS does NOT guarantee shortest path as it may take unnecessarily deep detours down wrong branches before finding the target, potentially using many more edges than necessary.

Q5. What happens if we don't use a VISITED array in DFS/BFS?

Without a visited array, graph traversal would enter an **infinite loop** for any graph containing a cycle. For example, in the graph $1-2-3-1$, DFS starting at 1 would go: $1 \rightarrow 2 \rightarrow 3 \rightarrow 1 \rightarrow 2 \rightarrow 3 \rightarrow \dots$ and never terminate.

The visited array ensures each vertex is processed **exactly once**, preventing redundant processing, infinite loops, and incorrect output. It is a **mandatory component** of both DFS and BFS algorithms.

Q6. What are the applications of DFS and BFS?

DFS Applications:

1. Maze solving — explores one path completely before trying another
2. Cycle detection in graphs
3. Topological sorting of directed graphs
4. Finding connected components
5. Solving puzzles with one solution (like Sudoku)

BFS Applications:

1. Shortest path in unweighted graphs
2. Social network analysis — finding friends within k degrees
3. Web crawlers — crawl the web level by level
4. GPS navigation — finding nearest locations
5. Broadcasting in networks — sending data to all nodes

🔑 MASTER MEMORY TRICKS — SESSION 4

STACK → DFS → DEEP → Recursion friendly

QUEUE → BFS → WIDE → Level-order → Shortest path

Inorder → In(middle) → Sorted BST output

Preorder → Pre(first) → Copy tree → PREFIX expression

Postorder → Post(last) → Delete tree → POSTFIX expression

📁 SESSION 5 & 6 OF 7

| Topics: Quick Sort + Merge Sort (Complete Package)

► TOPIC 12: QUICK SORT

◆ THEORY

Quick Sort is a highly efficient **divide and conquer** sorting algorithm. It works by selecting a special element called the **PIVOT** and **partitioning** the array around it — placing all smaller elements to the left of pivot and all larger elements to the right, then recursively sorting each side.

Core Idea: "Pick a pivot. Put everything smaller on its left. Put everything larger on its right. Recursively repeat on both halves."

Key Characteristics: Sorting strategy: **Divide and Conquer** — In-place sorting: Does **NOT** require extra array — **Unstable** sort — Recursive algorithm — Best/Average: **$O(n \log n)$** — Worst Case: **$O(n^2)$**

Real-world analogy: Imagine sorting a deck of cards. You pick one card as pivot (say, 7). You put all cards less than 7 in your left hand and all cards greater than 7 in your right hand. Then you repeat this process for each hand separately until everything is sorted.

◆ PIVOT SELECTION STRATEGIES

Strategy	Description	Risk
First element	Always pick $A[\text{low}]$ as pivot	$O(n^2)$ on sorted array
Last element	Always pick $A[\text{high}]$ as pivot	$O(n^2)$ on sorted array
Middle element	Pick $A[(\text{low}+\text{high})/2]$	Generally good
Random element	Pick random index	Avoids worst case
Median of three	Median of first, middle, last	Best practical choice

💡 For exam purposes: Teacher's algorithm uses **LAST element as pivot** — this is standard in most textbooks.

◆ ALGORITHMS

PARTITION Algorithm

```
Algorithm: PARTITION
Input: Array A, low, high
Output: Pivot index (pivot placed in correct position)

Steps:
1. Start
2. Set pivot = A[high]           ← last element as pivot
3. Set i = low - 1               ← boundary pointer
```

```

4. For j = low to high - 1:
    If A[j] <= pivot:
        i = i + 1
        Swap A[i] and A[j]      ← bring small element left
5. Swap A[i+1] and A[high]      ← place pivot in correct position
6. Return i + 1                 ← return pivot's final index
7. Stop

```

QUICKSORT Algorithm

```

Algorithm: QUICKSORT
Input: Array A, low, high
Output: Sorted Array

Steps:
1. Start
2. If low < high:
    a. pi = PARTITION(A, low, high)      ← base case: 1 or 0 elements = done
    b. QUICKSORT(A, low, pi - 1)        ← partition and get pivot index
    c. QUICKSORT(A, pi + 1, high)       ← sort left of pivot
    d. QUICKSORT(A, pi + 1, high)       ← sort right of pivot
3. Stop

```

Best/Average: $O(n \log n)$ — Worst Case: $O(n^2)$ — Space: $O(\log n)$ recursion stack

💡 Memory Trick: "PIVOT is the boss. i tracks the LEFT BOUNDARY. j scans everyone. If j finds someone $\leq$ boss, swap them to the left zone. At end, boss takes his rightful throne (swap with i+1)."

⚠ Critical conditions to remember:

- i starts at **low - 1** (one before the start)
- j loops from **low** to **high - 1** (NOT including pivot itself)
- Final swap places pivot at **i + 1**
- Return **i + 1** as partition index
- Recursive calls: left half = **(low, pi-1)**, right half = **(pi+1, high)**
- Base case: **low < high** (if $\text{low} \geq \text{high}$, subarray has 0 or 1 elements)

💎 COMPLETE DRY RUN: [10, 80, 30, 90, 40, 50, 70]

Indices: low=0, high=6

PASS 1: PARTITION(A, 0, 6) — pivot = A[6] = 70, i = -1

j	A[j]	A[j] ≤ 70?	Action	i	Array State
0	10	✓ Yes	i=0, swap A[0]↔A[0]	0	[10,80,30,90,40,50,70]
1	80	✗ No	No swap	0	[10,80,30,90,40,50,70]
2	30	✓ Yes	i=1, swap A[1]↔A[2]	1	[10,30,80,90,40,50,70]
3	90	✗ No	No swap	1	[10,30,80,90,40,50,70]
4	40	✓ Yes	i=2, swap A[2]↔A[4]	2	[10,30,40,90,80,50,70]
5	50	✓ Yes	i=3, swap A[3]↔A[5]	3	[10,30,40,50,80,90,70]

Loop ends. Swap pivot: swap A[i+1]=A[4] with A[high]=A[6]: **[10,30,40,50,70,90,80]**

pivot 70 is now at index 4 ✓ — Return $pi = 4$

Left of 70: [10,30,40,50] — all < 70 ✓ — Right of 70: [90,80] — all > 70 ✓

PASS 2: QUICKSORT(A, 0, 3) — Sort [10,30,40,50]

PARTITION(A, 0, 3), pivot = A[3] = 50, $i = -1$: All elements ≤ 50 , so they stay. 50 lands at index 3. $pi=3$.

QUICKSORT(A,0,2): Sort [10,30,40] → pivot=40 at index 2, $pi=2$.

QUICKSORT(A,0,1): Sort [10,30] → pivot=30, $10 \leq 30$ so 10 stays, 30 at index 1, $pi=1$.

QUICKSORT(A,0,0): low=high → **BASE CASE** → return.

Left half [10,30,40,50] fully sorted! ✓

PASS 3: QUICKSORT(A, 5, 6) — Sort [90,80]

PARTITION(A,5,6), pivot=A[6]=80, $i=4$: $90 > 80$, no swap. Final: swap A[5]↔A[6] → [80,90]. $pi=5$.

✓ **FINAL SORTED ARRAY: [10, 30, 40, 50, 70, 80, 90]**

◆ BEST VS WORST CASE EXPLANATION

Best Case — $O(n \log n)$: Pivot always divides the array into two equal halves. Each half is exactly $n/2 \rightarrow \log n$ levels of recursion. Each level processes n elements → Total: $n \times \log n$.

Worst Case — $O(n^2)$: Pivot is always the smallest or largest element (sorted array with last-element pivot). Each partition produces: left=[$n-1$ elements], right=[] → n levels of recursion → $n + (n-1) + \dots + 1 = O(n^2)$.

Worst case example: [1,2,3,4,5] with last-element pivot
Each partition: left=[1,2,3,4], right=[] → Skewed tree = $O(n^2)$

◆ EXAM QUESTIONS — QUICK SORT

Q1. What is Quick Sort? Explain its working principle with the concept of pivot and partitioning.

Quick Sort is a **divide and conquer** sorting algorithm that sorts an array by selecting a **pivot element** and **partitioning** the array into two sub-arrays: elements **less than or equal to** the pivot (go left) and elements **greater than** the pivot (go right).

After partitioning, the pivot is placed at its **final correct sorted position**. The algorithm then **recursively** applies the same process to the left and right sub-arrays independently.

Working principle step by step: (1) Select pivot (last element); (2) Rearrange array so elements $\leq$ pivot are on the left, $>$ pivot on the right; (3) Place pivot at its correct position (partition index); (4) Recursively sort left subarray (low to $pi-1$); (5) Recursively sort right subarray ($pi+1$ to high); (6) Base case: subarray with 0 or 1 element is already sorted.

Key insight: After each partition step, the pivot element is in its **permanently correct position** — it will never move again.

Q2. Complete dry run of Quick Sort on: [64, 25, 12, 22, 11]

PARTITION(A, 0, 4): pivot = 11, $i = -1$

j	A[j]	$\leq 11?$	Action	i

0	64	x	No swap	-1
1	25	x	No swap	-1
2	12	x	No swap	-1
3	22	x	No swap	-1

Swap $A[i+1]=A[0]$ with $A[4]$: swap 64↔11 → [11,25,12,22,64], pi=0. 11 at index 0 ✓

QUICKSORT(A,0,-1): low > high → return (base case)

QUICKSORT(A,1,4): Sort [25,12,22,64] → pivot=64, all $\leq$ 64, pi=4. 64 stays at index 4.

QUICKSORT(A,1,3): Sort [25,12,22] → pivot=22, i=0. 25>22 no swap. 12≤22: i=1, swap $A[1]↔A[2]$ → [11,12,25,22,64]. Swap $A[2]↔A[3]$: swap 25↔22 → [11,12,22,25,64], pi=2.

QUICKSORT(A,1,1): Single element [12] → return. QUICKSORT(A,3,3): Single element [25] → return.

✓ **FINAL SORTED ARRAY: [11, 12, 22, 25, 64]**

Q3. What is the time complexity of Quick Sort? When does worst case occur and how can it be avoided?

Case	Complexity	When it occurs
Best Case	$O(n \log n)$	Pivot always divides array into equal halves
Average Case	$O(n \log n)$	Random data, pivot divides reasonably
Worst Case	$O(n^2)$	Pivot is always smallest or largest element

Space Complexity: $O(\log n)$ for recursion stack (best/average), $O(n)$ worst case.

Why worst case is $O(n^2)$: When the input array is **already sorted** and we always pick the last element as pivot, each partition produces one subarray of $n-1$ elements and one empty subarray. This creates n levels of recursion, each doing $O(n)$ work.

How to avoid worst case: (1) **Random pivot selection**; (2) **Median-of-three** — use median of first, middle, and last elements; (3) **Shuffle the array** before sorting.

Q4. Is Quick Sort in-place? What does in-place mean?

Yes, Quick Sort is an **in-place** sorting algorithm. **In-place** means the algorithm sorts the array using **only the original array's memory** — it does not require creating a separate/extra array to hold elements during sorting.

Quick Sort only uses: temporary variable during swaps + recursion call stack: $O(\log n)$ space. This is a major advantage over Merge Sort which requires **$O(n)$ extra space**.

Q5. What is the role of the PARTITION function in Quick Sort?

The PARTITION function is the **core engine** of Quick Sort. It performs three critical tasks:

1. **Selects the pivot** (typically the last element $A[\text{high}]$)
2. **Rearranges the array** so that all elements $\leq$ pivot are moved to the LEFT and all elements $>$ pivot

are moved to the RIGHT

3. **Places the pivot** at its final correct sorted position and **returns that index (pi)**

After PARTITION executes, the pivot element is **permanently placed** at its correct position — it will never need to move again. The quality of pivot selection in PARTITION directly determines Quick Sort's performance.

Q6. What is divide and conquer? How does Quick Sort apply it?

A **Divide and Conquer** algorithm: (1) **Divide** — break problem into smaller subproblems; (2) **Conquer** — recursively solve each subproblem; (3) **Combine** — merge the solutions.

Quick Sort applies it as:

- **Divide:** PARTITION selects pivot and divides array into two sub-arrays
- **Conquer:** Recursively apply QUICKSORT to both sub-arrays
- **Combine:** No explicit combine step needed — partitioning places pivot correctly, both sub-arrays sorted in-place, full array sorted automatically

Notable: Unlike Merge Sort, Quick Sort does its heavy work in the **Divide step** (partition), not the Combine step.

Q7. What is the base case in Quick Sort recursion and why is it important?

The **base case** in Quick Sort is: **if low $\geq$ high $\rightarrow$ return** (do nothing).

This condition means the current subarray has **0 or 1 elements**, which is already sorted by definition. Without the base case, the recursive calls would continue indefinitely — eventually calling with the same indices repeatedly, causing **infinite recursion** and a **stack overflow**.

► TOPIC 13: MERGE SORT

◆ THEORY

Merge Sort is a **divide and conquer** sorting algorithm that works by recursively **splitting** the array into halves until each half has only one element (trivially sorted), then **merging** those halves back together in sorted order.

Core Idea: "Keep splitting the array in half until you have single elements. Then merge them back in sorted order."

Key Characteristics: Sorting strategy: **Divide and Conquer** — **Stable** sort — **NOT in-place** (requires $O(n)$ extra space) — Always **$O(n \log n)$** — Excellent for **linked lists** and **external sorting**.

◆ HOW MERGE SORT WORKS — TWO PHASES

Phase 1: DIVIDE

[38, 27, 43, 3, 9, 82, 10]


```

      ↓ split
[38, 27, 43, 3]    [9, 82, 10]
      ↓ split      ↓ split
[38, 27] [43, 3]  [9, 82] [10]
      ↓          ↓          ↓
[38] [27] [43] [3] [9] [82] [10]
← All single elements — all "sorted" by definition

```

Phase 2: MERGE (CONQUER)

```

[38] [27] → [27, 38]      [43] [3] → [3, 43]
[9] [82] → [9, 82]      [10] → [10]
[27, 38] [3, 43] → [3, 27, 38, 43]
[9, 82] [10] → [9, 10, 82]
[3, 27, 38, 43] [9, 10, 82] → [3, 9, 10, 27, 38, 43, 82] ✓

```

◆ ALGORITHMS

MERGE Algorithm (The Core Step)

```

Algorithm: MERGE
Input: Array A, left, mid, right
Output: Merged sorted subarray

Steps:
1. Start
2. Create temp array L[0..n1] where n1 = mid - left + 1
3. Create temp array R[0..n2] where n2 = right - mid
4. Copy A[left..mid] into L[]      ← copy left half
5. Copy A[mid+1..right] into R[]  ← copy right half
6. Set i = 0, j = 0, k = left
7. While i < n1 AND j < n2:
    If L[i] <= R[j]:
        A[k] = L[i]; i = i + 1
    Else:
        A[k] = R[j]; j = j + 1
    k = k + 1
8. While i < n1:                    ← copy remaining left elements
    A[k] = L[i]; i = i + 1; k = k + 1
9. While j < n2:                    ← copy remaining right elements
    A[k] = R[j]; j = j + 1; k = k + 1
10. Stop

```

MERGESORT Algorithm

```

Algorithm: MERGESORT
Input: Array A, left, right
Output: Sorted Array

Steps:
1. Start
2. If left < right:                ← base case: single element, return
    a. mid = (left + right) / 2    ← find middle point
    b. MERGESORT(A, left, mid)     ← sort left half
    c. MERGESORT(A, mid + 1, right) ← sort right half
    d. MERGE(A, left, mid, right)  ← merge the two halves

```

Best/Average/Worst: $O(n \log n)$ ALWAYS — Space: $O(n)$

💡 Trick: "SPLIT until SINGLE. MERGE back SORTED. $\text{Mid} = (\text{left} + \text{right}) / 2$. In MERGE: compare front of both halves, pick smaller, advance pointer."

⚠ Critical conditions:

- Base case: **left < right** (if $\text{left} \geq \text{right}$, 0 or 1 element — return)
- Mid formula: **$\text{mid} = (\text{left} + \text{right}) / 2$** (integer division)
- Recursive calls: **left = (left, mid)**, **right = (mid+1, right)**
- After merge loop: **copy remaining elements** from whichever half isn't exhausted
- Merge uses **extra arrays L and R** — this is why space is $O(n)$

◆ COMPLETE STEP-BY-STEP DRY RUN: [38, 27, 43, 3]

left=0, right=3, mid=1

MERGESORT(A,0,1): Sort [38,27]

mid=0. MERGESORT(A,0,0) and MERGESORT(A,1,1) both BASE CASE. Then MERGE(A,0,0,1):

L=[38], R=[27], i=0, j=0, k=0

Step	L[i] vs R[j]	Winner	A[k]	i	j	k
1	38 vs 27	R[0]=27 smaller	A[0]=27	0	1	1

j exhausted. Remaining L: L[0]=38 → A[1]=38. **After MERGE: [27, 38, 43, 3]** ✓

MERGESORT(A,2,3): Sort [43,3]

mid=2. BASE CASEs for [43] and [3]. Then MERGE(A,2,2,3):

L=[43], R=[3], i=0, j=0, k=2

Step	L[i] vs R[j]	Winner	A[k]
1	43 vs 3	R[0]=3 smaller	A[2]=3

j exhausted. Remaining L: 43 → A[3]=43. **After MERGE: [27, 38, 3, 43]** ✓

MERGE(A, 0, 1, 3): Merge [27,38] and [3,43]

L=[27,38], R=[3,43], i=0, j=0, k=0

Step	L[i] vs R[j]	Winner	A[k]
1	27 vs 3	3	A[0]=3
2	27 vs 43	27	A[1]=27
3	38 vs 43	38	A[2]=38

i exhausted. Remaining R: 43 → A[3]=43.

✓ **FINAL SORTED ARRAY: [3, 27, 38, 43]**

◆ LARGER DRY RUN: [5, 2, 4, 6, 1, 3]

Division Phase:

```
Level 0:    [5, 2, 4, 6, 1, 3]
            ↓ mid=2
Level 1:    [5,2,4]          [6,1,3]
            ↓ mid=1          ↓ mid=4 (index)
Level 2:    [5,2] [4]       [6,1] [3]
            ↓             ↓
Level 3:    [5] [2] [4]     [6] [1] [3]
← All single elements
```

Merge Phase:

```
[5] [2] → [2,5]
[2,5] [4] → [2,4,5]
[6] [1] → [1,6]
[1,6] [3] → [1,3,6]
[2,4,5] [1,3,6] → [1,2,3,4,5,6] ✓
```

Final MERGE trace — [2,4,5] and [1,3,6]:

Compare	Winner	Array position
2 vs 1	1	A[0]=1
2 vs 3	2	A[1]=2
4 vs 3	3	A[2]=3
4 vs 6	4	A[3]=4
5 vs 6	5	A[4]=5
remaining R	6	A[5]=6

✓ **RESULT: [1, 2, 3, 4, 5, 6]**

◆ COMPLEXITY ANALYSIS — WHY $O(n \log n)$?

Number of levels of recursion: Array of size n is halved at each level → Number of levels = $\log_2(n)$

Work done at each level: At every level, all n elements are processed during merging → Each level = $O(n)$ work

Total: $O(n) \times O(\log n) = O(n \log n)$

```
Level 0: n elements → O(n) merge work
Level 1: 2 × n/2 elements → O(n) merge work
Level 2: 4 × n/4 elements → O(n) merge work
...
Level log n: n × 1 elements → O(n) merge work
Total levels = log n → Total = n × log n = O(n log n)
```

💡 **This is always $O(n \log n)$ regardless of input — Merge Sort has no bad input!**

◆ EXAM QUESTIONS — MERGE SORT

Q1. What is Merge Sort? Explain the two phases: divide and merge.

Merge Sort is a **divide and conquer** sorting algorithm that operates in two distinct phases:

Phase 1 — DIVIDE: The algorithm recursively splits the array into two halves by finding the midpoint: $\text{mid} = (\text{left} + \text{right}) / 2$. This continues until subarrays of size 1 are reached. A single element is trivially sorted — this is the **base case** ($\text{left} \geq \text{right} \rightarrow \text{return}$).

Phase 2 — MERGE (Conquer): The algorithm merges pairs of sorted subarrays back into larger sorted arrays. The MERGE operation compares elements from two sorted halves one by one, always picking the smaller element first, producing a single merged sorted array. Remaining elements from either half are appended after one half is exhausted.

Q2. Dry run Merge Sort on: [12, 11, 13, 5, 6, 7]

Division: [12,11,13,5,6,7] $\rightarrow$ [12,11,13] and [5,6,7] $\rightarrow$ [12,11] [13] and [5,6] [7] $\rightarrow$ [12][11] [13] [5][6] [7]

MERGE [12][11]: L=[12], R=[11] $\rightarrow 12 > 11$: pick 11 $\rightarrow A[0]=11$; remaining: 12 $\rightarrow A[1]=12$. **Result:** [11,12]

MERGE [11,12][13]: $11 < 13$: pick 11; $12 < 13$: pick 12; remaining: 13. **Result:** [11,12,13]

MERGE [5][6]: $5 < 6$: pick 5; remaining: 6. **Result:** [5,6]

MERGE [5,6][7]: $5 < 7$: pick 5; $6 < 7$: pick 6; remaining: 7. **Result:** [5,6,7]

FINAL MERGE [11,12,13] and [5,6,7]:

L[i] vs R[j]	Winner	Position
11 vs 5	5	A[0]=5
11 vs 6	6	A[1]=6
11 vs 7	7	A[2]=7
remaining L: 11,12,13	11,12,13	A[3]=11, A[4]=12, A[5]=13

✓ **SORTED: [5, 6, 7, 11, 12, 13]**

Q3. Why is Merge Sort always $O(n \log n)$ while Quick Sort can be $O(n^2)$?

Merge Sort always achieves $O(n \log n)$ because: It always splits the array into **exactly two equal halves** ($\text{mid} = \text{left} + \text{right} / 2$). This produces exactly **$\log_2(n)$ levels** of recursion — always, regardless of data. The input data arrangement has absolutely no effect on Merge Sort's splitting strategy.

Quick Sort degrades to $O(n^2)$ because: Its split depends on **pivot selection**. If the pivot is always the smallest or largest element (sorted input with last-element pivot), one subarray gets all $n-1$ elements and the other gets 0. This creates **n levels** of recursion instead of $\log n$.

Bottom line: Merge Sort's division is **data-independent** (always halves). Quick Sort's division is **data-dependent** (depends on pivot quality).

Q4. What is the space complexity of Merge Sort? Why does it need extra space?

Merge Sort has a **space complexity of $O(n)$** for the auxiliary (temporary) arrays used during the merge step.

Why extra space is needed: When merging two sorted subarrays, we cannot merge them in-place without losing data. For example, merging [3,5] and [1,4] requires placing 1 first, but index 0 already holds 3. If we overwrite 3, we lose it. Therefore, Merge Sort copies each half into **temporary arrays L and R**, merges from those into the original array.

Comparison: Quick Sort only uses $O(\log n)$ stack space, making it more space-efficient. This is a key disadvantage of Merge Sort — the price paid for guaranteed $O(n \log n)$ performance.

Q5. Is Merge Sort stable? What does stability mean?

Yes, Merge Sort is a **stable** sorting algorithm. **Stability** means that when two elements have **equal values**, their **relative order in the original array is preserved** in the sorted output.

Why Merge Sort is stable: In the MERGE step, when $L[i] == R[j]$, the algorithm always picks **$L[i]$ first** (the condition is $L[i] \leq R[j]$). Since L comes from the left (earlier in original array) and R from the right, this preserves the original relative order of equal elements.

Quick Sort is NOT stable — swaps can change the relative order of equal elements.

Q6. When would you prefer Merge Sort over Quick Sort and vice versa?

Prefer Merge Sort when:

- Guaranteed $O(n \log n)$ performance is required (cannot afford worst case)
- Sorting a **linked list** (Merge Sort works well without random access)
- **Stability** is required (preserving order of equal elements)
- **External sorting** — data is too large for RAM

Prefer Quick Sort when:

- **Memory is limited** (Quick Sort is in-place, $O(\log n)$ space vs $O(n)$)
- **Average performance** is the priority (smaller constant factors in practice)
- Data is **random** (worst case is unlikely)
- Sorting **arrays** in memory where cache performance matters

Q7. MASTER COMPARISON: Quick Sort vs Merge Sort

Feature	Quick Sort	Merge Sort
Strategy	Divide and Conquer	Divide and Conquer
Best Case	$O(n \log n)$	$O(n \log n)$
Average Case	$O(n \log n)$	$O(n \log n)$
Worst Case	$O(n^2)$	$O(n \log n)$
Space Complexity	$O(\log n)$ recursion stack	$O(n)$ extra arrays

In-place?	✓ Yes	✗ No
Stable?	✗ No	✓ Yes
Divide step	Unequal (pivot-based)	Equal halves (mid)
Main work in	Divide (partition)	Conquer (merge)
Worst case input	Already sorted array	None (always $n \log n$)
Best for	Arrays, average case	Linked lists, guaranteed

🔗 EXAM STRATEGY — QUICK SORT & MERGE SORT

1. For Quick Sort dry run: Always show the full partition table with columns: j, A[j], condition, action, i, array state
2. For Merge Sort dry run: Draw the division tree first, then show each merge step separately
3. Always state all three time complexities (best, average, worst) for both algorithms
4. Quick Sort worst case question is almost certain — explain with sorted input example
5. Merge Sort extra space question is almost certain — explain why copying is needed
6. Stable vs Unstable: Merge Sort=Stable, Quick Sort=Unstable — always mention!

🔑 ULTIMATE SORTING FORMULA CARD

QUICK SORT:

```
Pivot = last element (A[high])
Partition: i = low-1, j scans low to high-1
If A[j] <= pivot: i++, swap A[i] and A[j]
Final: swap A[i+1] with A[high], return i+1
Recurse: QUICKSORT(low, pi-1) and QUICKSORT(pi+1, high)
Complexity:  $O(n \log n)$  avg,  $O(n^2)$  worst
Space:  $O(\log n)$  | In-place: YES | Stable: NO
```

MERGE SORT:

```
Split: mid = (left + right) / 2
Recurse: MERGESORT(left, mid) and MERGESORT(mid+1, right)
Merge: Copy to L[] and R[], compare L[i] vs R[j], pick smaller
Copy remaining elements after one half is exhausted
Complexity:  $O(n \log n)$  ALWAYS
Space:  $O(n)$  | In-place: NO | Stable: YES
```

SESSION 7 (FINAL)

| Topics: Infix / Prefix / Postfix + Complete Revision Package

► TOPIC 14: INFIX, PREFIX & POSTFIX EXPRESSIONS

◆ THEORY — WHAT ARE EXPRESSIONS?

In mathematics and computing, an **arithmetic expression** consists of: **Operands** — numbers or variables (e.g., A, B, 5, 10); **Operators** — arithmetic symbols (+, -, *, /, ^); **Parentheses** — to control evaluation order.

There are **three ways** to write the same expression depending on **where the operator is placed** relative to its operands.

◆ THE THREE NOTATIONS

1. INFIX Notation — Operator placed **BETWEEN** two operands

```
A + B          (A plus B)
(A + B) * C
A + B * C - D
```

- This is the **normal human-readable** form
- Requires **parentheses** and **precedence rules** to resolve ambiguity
- Example: $A + B * C$ means $B * C$ first (precedence), not $A + B$ first

2. PREFIX Notation (Polish Notation) — Operator placed **BEFORE** its operands

```
+ A B          (means A + B)
* + A B C      (means (A+B) * C)
- + A * B C D  (means A + B * C - D)
```

- No parentheses needed — **unambiguous by structure**
- Evaluated **right to left**
- Also called **Polish Notation**

3. POSTFIX Notation (Reverse Polish Notation) — Operator placed **AFTER** its operands

```
A B +          (means A + B)
A B + C *      (means (A+B) * C)
A B C * + D -  (means A + B * C - D)
```

- No parentheses needed — **unambiguous by structure**
- Evaluated **left to right**
- Also called **Reverse Polish Notation (RPN)**
- **Most important for computers** — used in stack-based evaluation and compilers

◆ OPERATOR PRECEDENCE & ASSOCIATIVITY (CRITICAL — MUST MEMORIZE)

Precedence	Operator	Associativity
Highest (3)	^ (exponent)	Right to Left
Medium (2)	* / (multiply, divide)	Left to Right
Lowest (1)	+ - (add, subtract)	Left to Right
Brackets	()	Highest — always first

◆ Memory Trick: "**BODMAS / PEMDAS**" — Brackets, Exponents, Multiply/Divide, Add/Subtract.

◆ QUICK CONVERSION REFERENCE TABLE

Infix	Prefix	Postfix
A + B	+ A B	A B +
A + B - C	- + A B C	A B + C -
(A + B) * C	* + A B C	A B + C *
A * B + C * D	+ * A B * C D	A B * C D * +
A + B * C	+ A * B C	A B C * +
(A - B) * (C + D)	* - A B + C D	A B - C D + *

◆ ALGORITHM: INFIX TO POSTFIX CONVERSION

```
Algorithm: INFIX_TO_POSTFIX
Input: Infix expression string
Output: Postfix expression string

Steps:
1. Start
2. Create empty STACK and empty OUTPUT string
3. Scan the infix expression LEFT to RIGHT, one token at a time:

   For each token:

   a. If token is an OPERAND (letter/number):
      → Append directly to OUTPUT

   b. If token is '(' (left parenthesis):
      → PUSH onto STACK

   c. If token is ')' (right parenthesis):
      → POP from STACK and append to OUTPUT
      until '(' is encountered
      → Discard the '(' (do NOT add to output)

   d. If token is an OPERATOR (+, -, *, /, ^):
      → While STACK is not empty
        AND top of STACK is not '('
        AND precedence(top of STACK) >= precedence(token):
```



```

    → POP from STACK and append to OUTPUT
    → PUSH current operator onto STACK

4. After scanning all tokens:
    → While STACK is not empty:
        → POP and append to OUTPUT

5. OUTPUT is the Postfix expression
6. Stop

```

🧠 Memory Trick:

"Operand → Output directly.

(→ Push always.

) → Pop until matching (.

Operator → Pop higher/equal precedence first, then Push.

End → Pop everything remaining."

⚠ Critical rules:

- Left parenthesis on stack has LOWEST priority — never pop it for an operator
- Right parenthesis is NEVER pushed to stack — it triggers popping
- For equal precedence, pop before pushing (left-to-right associativity)
- Exponent (^) is right-associative — do NOT pop equal precedence ^ from stack

♦ INFIX TO POSTFIX — DRY RUNS

Example 1: $A + B * C$

Token	Action	Stack	Output
A	Operand → Output	[]	A
+	Stack empty → Push	[+]	A
B	Operand → Output	[+]	A B
*	$\text{prec}(*) > \text{prec}(+) \rightarrow \text{Push}$	[+, *]	A B
C	Operand → Output	[+, *]	A B C
End	Pop all: *, then +	[]	A B C * +

Postfix: $A B C * +$ — Meaning: $A + (B * C)$ ✓ Precedence respected!

Example 2: $(A + B) * C$

Token	Action	Stack	Output
(	Push (	[(]	
A	Operand	[(]	A
+	top is '(' → Push	[(, +]	A
B	Operand	[(, +]	A B
)	Pop until (: pop +, discard (	[]	A B +
*	Stack empty → Push	[*]	A B +
C	Operand	[*]	A B + C

End	Pop all: *	[]	A B + C *
-----	------------	----	-----------

Postfix: A B + C * — Meaning: (A+B)*C ✓

Example 3: A + B * C - D

Token	Action	Stack	Output
A	Operand	[]	A
+	Push	[+]	A
B	Operand	[+]	A B
*	prec(*) > prec(+) → Push	[+, *]	A B
C	Operand	[+, *]	A B C
-	prec(-) ≤ prec(*): Pop *; prec(-) ≤ prec(+): Pop +; Push -	[-]	A B C * +
D	Operand	[-]	A B C * + D
End	Pop -	[]	A B C * + D -

Postfix: A B C * + D - — Meaning: (A + B*C) - D ✓

Example 4: (A + B) * (C - D)

Token	Action	Stack	Output
(	Push	[(]	
A	Operand	[(]	A
+	top is (→ Push	[(, +]	A
B	Operand	[(, +]	A B
)	Pop until (: pop +	[]	A B +
*	Push	[*]	A B +
(	Push	[* , (]	A B +
C	Operand	[* , (]	A B + C
-	top is (→ Push	[* , (, -]	A B + C
D	Operand	[* , (, -]	A B + C D
)	Pop until (: pop -	[*]	A B + C D -
End	Pop *	[]	A B + C D - *

Postfix: A B + C D - * — Meaning: (A+B)*(C-D) ✓

◆ ALGORITHM: POSTFIX EVALUATION

```

Algorithm: EVALUATE_POSTFIX
Input: Postfix expression
Output: Result of expression

```

```

Steps:
1. Start
2. Create empty STACK
3. Scan postfix expression LEFT to RIGHT:

   For each token:

   a. If token is an OPERAND (number):
      → PUSH onto STACK

   b. If token is an OPERATOR (+, -, *, /, ^):
      → POP top element → call it B (second operand)
      → POP next element → call it A (first operand)
      → Compute result = A operator B
      → PUSH result onto STACK

4. After scanning all tokens:
   → POP the final value from STACK = RESULT
5. Stop

```

💡 Trick: "Number → Push. Operator → Pop B (top), Pop A (below B), Compute A op B, Push result."

⚠️ CRITICAL ORDER: Pop B first (top), then Pop A. Apply as **A operator B** (NOT B operator A).
 For + and *: order doesn't matter (commutative). For - and /: ORDER MATTERS! $A-B \neq B-A$.
First popped = B (right operand), Second popped = A (left operand)

◆ POSTFIX EVALUATION — DRY RUNS

Example 1: 5 3 + 2 * (means $(5+3)*2 = 16$)

Token	Action	Stack
5	Push 5	[5]
3	Push 3	[5, 3]
+	Pop B=3, Pop A=5, $5+3=8$, Push 8	[8]
2	Push 2	[8, 2]
*	Pop B=2, Pop A=8, $8*2=16$, Push 16	[16]
End	Pop result	Result = 16

Example 2: 6 2 3 + - (means $6-(2+3) = 1$)

Token	Action	Stack
6	Push 6	[6]
2	Push 2	[6, 2]
3	Push 3	[6, 2, 3]
+	Pop B=3, Pop A=2, $2+3=5$, Push 5	[6, 5]
-	Pop B=5, Pop A=6, $6-5=1$, Push 1	[1]
End	Pop result	Result = 1

Example 3: 2 3 1 * + 9 - (means $2+3*1-9 = -4$)

Token	Action	Stack
2	Push 2	[2]
3	Push 3	[2, 3]
1	Push 1	[2, 3, 1]
*	Pop B=1, Pop A=3, $3*1=3$, Push 3	[2, 3]
+	Pop B=3, Pop A=2, $2+3=5$, Push 5	[5]
9	Push 9	[5, 9]
-	Pop B=9, Pop A=5, $5-9=-4$, Push -4	[-4]
End	Pop result	Result = -4

◆ INFIX TO PREFIX CONVERSION

Method: Reverse the infix expression, convert to postfix (with reversed parentheses), then reverse the result.

Steps: (1) **Reverse** the infix expression; (2) **Swap** every '(' with ')' and vice versa; (3) Apply **Infix to Postfix** algorithm; (4) **Reverse** the resulting postfix expression; (5) That is your **Prefix** expression.

Example: Convert $(A + B) * C$ to Prefix

Original: $(A+B)*C \rightarrow$ Reverse char by char: $C*)B+A(\rightarrow$ Swap brackets: $C*(B+A)$

Apply Infix→Postfix on $C*(B+A)$:

Token	Action	Stack	Output
C	Operand	[]	C
*	Push	[*]	C
(	Push	[* , (]	C
B	Operand	[* , (]	C B
+	top is ($\rightarrow$ Push	[* , (, +]	C B
A	Operand	[* , (, +]	C B A
)	Pop until (: pop +	[*]	C B A +
End	Pop *	[]	C B A + *

Postfix result: $C B A + * \rightarrow$ Reverse: $* + A B C$

Prefix: $* + A B C$ — Meaning: $(A+B)*C$ ✓

◆ PREFIX EVALUATION (Right to Left Scan)

Algorithm: EVALUATE_PREFIX

Input: Prefix expression

Output: Result

Steps:

1. Start

```

2. Create empty STACK
3. Scan prefix expression RIGHT TO LEFT:
  a. If token is OPERAND → PUSH onto STACK
  b. If token is OPERATOR:
      → POP A (first pop = left operand)
      → POP B (second pop = right operand)
      → Compute A operator B
      → PUSH result
4. POP final result from STACK
5. Stop

```

💡 Trick: Prefix evaluation = scan RIGHT to LEFT (opposite of postfix). First pop = A (left operand), second pop = B (right operand).

◆ COMPLETE NOTATION COMPARISON TABLE

Feature	Infix	Prefix	Postfix
Operator position	Between operands	Before operands	After operands
Also called	Standard	Polish Notation	Reverse Polish Notation
Parentheses needed?	✓ Yes	✗ No	✗ No
Human readable?	✓ Yes	✗ Hard	✗ Hard
Computer efficient?	✗ Complex to parse	✓ Yes	✓ Yes (most common)
Evaluation direction	With precedence rules	Right to Left	Left to Right
Stack used for?	Operators (conversion)	Operands (evaluation)	Operands (evaluation)
Example	$A + B * C$	$+ A * B C$	$A B C * +$

◆ EXPRESSION TREE CONNECTION

```

Tree for: A + B * C
      +
     / \
    /   *
   /     \
  /       B C

```

- **Inorder** traversal → $A + B * C$ (Infix)
- **Preorder** traversal → $+ A * B C$ (Prefix)
- **Postorder** traversal → $A B C * +$ (Postfix)

💡 This is why tree traversals matter — they directly give expression notations!

◆ EXAM QUESTIONS — INFIX, PREFIX, POSTFIX

Q1. What is Infix, Prefix, and Postfix notation? Give one example of each for $A + B$.

Infix Notation: Operator placed **between** its two operands. Standard mathematical notation. Requires operator precedence rules and parentheses. Example: $A + B$

Prefix Notation (Polish Notation): Operator placed **before** its two operands. No parentheses needed.

Example: $+ A B$

Postfix Notation (Reverse Polish Notation): Operator placed **after** its two operands. No parentheses needed. Most suitable for computer evaluation using a stack. Example: $A B +$

All three represent the same operation: adding A and B.

Q2. Convert the following infix expressions to postfix: (a) $A * B + C$ (b) $A + B * C + D$ (c) $(A + B) * (C + D)$

(a) $A * B + C$

Token	Action	Stack	Output
A	Operand	[]	A
*	Push	[*]	A
B	Operand	[*]	A B
+	$\text{prec}(+) < \text{prec}(\ast) \rightarrow \text{Pop } \ast; \text{Push } +$	[+]	A B *
C	Operand	[+]	A B * C
End	Pop +	[]	A B * C +

Postfix: $A B * C +$ (means $(A*B)+C$) ✓

(b) $A + B * C + D$: Postfix: $A B C * + D +$ (means $(A+B*C)+D$) ✓

(c) $(A + B) * (C + D)$: Postfix: $A B + C D + *$ (means $(A+B)*(C+D)$) ✓

Q3. Evaluate the following postfix expressions: (a) $4 2 + 3 *$ (b) $5 1 2 + 4 * + 3 -$ (c) $3 4 + 2 * 7 /$

(a) $4 2 + 3 *$

Token	Action	Stack
4	Push	[4]
2	Push	[4,2]
+	$B=2, A=4, 4+2=6$	[6]
3	Push	[6,3]
*	$B=3, A=6, 6*3=18$	[18]

Result: 18 ✓

(b) $5 1 2 + 4 * + 3 -$: $1+2=3 \rightarrow 3*4=12 \rightarrow 5+12=17 \rightarrow 17-3=14$ ✓

(c) $3 4 + 2 * 7 /$: $3+4=7 \rightarrow 7*2=14 \rightarrow 14/7=2$ ✓

Q4. Convert $A * (B + C) / D$ to both Prefix and Postfix.

Postfix of $A * (B + C) / D$:

Token trace: A → output A; * → push; (→ push; B → output B; + → push (top is ()); C → output C;) → pop +: output "A B C +"; / → prec(/)=prec(*): pop *, push /: output "A B C + *"; D → output D; End: pop / → output "A B C + * D /"

Postfix: $A B C + * D /$

Prefix of $A * (B + C) / D$: Reverse: $D/(C+B)*A$ → Apply postfix: $D C B + / A *$ → Reverse: $/ * A + B C D$

Q5. What is the role of STACK in Infix to Postfix conversion? Why is a stack specifically used?

The stack plays a **central role** in Infix to Postfix conversion by temporarily **holding operators** until the right moment to output them.

Why stack is needed: In infix notation, an operator cannot be output immediately when encountered because a later operator might have higher precedence and should be applied first. The stack allows us to **defer the output** of operators.

Why not another structure? Stack is used because operator precedence creates a **LIFO (Last In, First Out)** behavior — the most recently pushed high-precedence operator is always the first to be output. A queue or array would not naturally support this precedence-based ordering.

In summary: Stack automatically handles precedence by ensuring higher-priority operators are processed before lower-priority ones, without needing parentheses.

Q6. Why does postfix not need parentheses but infix does?

Infix needs parentheses because the operator position (between operands) is ambiguous. For example, $A + B * C$ could mean $(A+B)*C$ or $A+(B*C)$. Without parentheses or precedence rules, the expression is ambiguous.

Postfix does not need parentheses because the order of operators in the expression **itself** encodes the evaluation sequence.

- $(A+B)*C$ in postfix: $A B + C *$
- $A+(B*C)$ in postfix: $A B C * +$

The two postfix expressions are visually different — no ambiguity exists. This is why **compilers convert infix to postfix** — it is far easier and more efficient to evaluate.

Q7. What is the difference between Prefix and Postfix evaluation?

Feature	Postfix Evaluation	Prefix Evaluation
Scan direction	Left to Right	Right to Left
When operand found	Push onto stack	Push onto stack
When operator found	Pop B (top), Pop A, compute A op B	Pop A (top), Pop B, compute A op B
Pop order	First pop = right operand (B)	First pop = left operand (A)

Final result	Pop from stack	Pop from stack
Ease of implementation	Easier (natural left-to-right)	Slightly tricky (right-to-left scan)

Q8. What are the practical applications of Postfix notation?

1. **Compiler Design:** Compilers internally convert infix expressions to postfix for efficient code generation. Postfix is easier to evaluate algorithmically.
2. **Expression Evaluation in Calculators:** Many scientific calculators (HP calculators) use Reverse Polish Notation (postfix) directly, eliminating the need for parentheses.
3. **Stack-Based Virtual Machines:** Java Virtual Machine (JVM) and Python's bytecode use postfix-like stack-based evaluation for arithmetic operations.
4. **Syntax Analysis:** Parsing arithmetic expressions in interpreters and programming languages relies on converting to postfix for evaluation.
5. **Postfix is unambiguous:** Since no parentheses or precedence rules are required, it simplifies computer parsing dramatically.

Q9. Convert infix: $(A - B / C) * (A / K - L)$ to postfix.

Token	Action	Stack	Output
(	Push	[(]	
A	Operand	[(]	A
-	top is (→ Push	[(, -]	A
B	Operand	[(, -]	A B
/	prec(/) > prec(-) → Push	[(, -, /]	A B
C	Operand	[(, -, /]	A B C
)	Pop until (: pop /, pop -	[]	A B C / -
*	Push	[*]	A B C / -
(	Push	[* , (]	A B C / -
A	Operand	[* , (]	A B C / - A
/	top is (→ Push	[* , (, /]	A B C / - A
K	Operand	[* , (, /]	A B C / - A K
-	prec(-) < prec(/) → Pop /; top is (→ Push -	[* , (, -]	A B C / - A K /
L	Operand	[* , (, -]	A B C / - A K / L
)	Pop until (: pop -	[*]	A B C / - A K / L -
End	Pop *	[]	A B C / - A K / L - *

Postfix: A B C / - A K / L - *

Q10. Trace: $A \wedge B * C - D + E / F / (G + H)$ to postfix.

Token	Action	Stack	Output
A	Operand	[]	A
$\wedge$	Push (highest prec)	[$\wedge$]	A
B	Operand	[$\wedge$]	A B
*	$\text{prec}(\ast) < \text{prec}(\wedge) \rightarrow \text{Pop } \wedge; \text{Push } \ast$	[*]	A B $\wedge$
C	Operand	[*]	A B $\wedge$ C
-	$\text{prec}(-) < \text{prec}(\ast) \rightarrow \text{Pop } \ast; \text{Push } -$	[-]	A B $\wedge$ C *
D	Operand	[-]	A B $\wedge$ C * D
+	$\text{prec}(+) = \text{prec}(-) \rightarrow \text{Pop } -; \text{Push } +$	[+]	A B $\wedge$ C * D -
E	Operand	[+]	A B $\wedge$ C * D - E
/	$\text{prec}(/) > \text{prec}(+) \rightarrow \text{Push}$	[+, /]	A B $\wedge$ C * D - E
F	Operand	[+, /]	A B $\wedge$ C * D - E F
/	$\text{prec}(/) = \text{prec}(/) \rightarrow \text{Pop } /; \text{Push } /$	[+, /]	A B $\wedge$ C * D - E F /
(	Push	[+, /, (]	A B $\wedge$ C * D - E F /
G	Operand	[+, /, (]	A B $\wedge$ C * D - E F / G
+	top is ($\rightarrow$ Push	[+, /, (, +]	A B $\wedge$ C * D - E F / G
H	Operand	[+, /, (, +]	A B $\wedge$ C * D - E F / G H
)	Pop until (: pop +	[+, /]	A B $\wedge$ C * D - E F / G H +
End	Pop /, pop +	[]	A B $\wedge$ C * D - E F / G H + / +

Postfix: $A B \wedge C * D - E F / G H + / +$

🔗 EXAM STRATEGY — INFIX/PREFIX/POSTFIX

1. **Always make a table** with columns: Token, Action, Stack, Output — for every conversion dry run
2. **State the precedence order** before starting any conversion — shows you know the rules
3. For evaluation, **always show each pop and computation** — never skip steps
4. **Parentheses rule** is the most tested: (always pushed,) triggers popping until matching (
5. **Order of pops** in evaluation: first pop = B (right), second pop = A (left) — for - and / this is CRITICAL
6. For complex expressions — go slow, one token at a time

FINAL REVISION PACKAGE

All Algorithms + Complexity Tables + Differences + Most Probable Questions + Cheat Sheet

► MASTER ALGORITHM REFERENCE — All Algorithms in One Place

◆ ARRAYS

```
TRAVERSE: For i=0 to n-1 → Print A[i]    [O(n)]
```

```
INSERT(pos, X):  
  If pos < 0 OR pos > n → "Invalid"  
  For i = n-1 downto pos → A[i+1] = A[i]  
  A[pos] = X;  n = n + 1    [O(n)]
```

```
DELETE(pos):  
  If pos < 0 OR pos >= n → "Invalid"  
  For i = pos to n-2 → A[i] = A[i+1]  
  n = n - 1    [O(n)]
```

```
SEARCH(X):  
  For i = 0 to n-1 → If A[i] == X return i  
  Return -1    [O(n)]
```

◆ SINGLY LINKED LIST

```
INSERT_BEGIN:  N.next = HEAD;  HEAD = N    [O(1)]
```

```
INSERT_END:    Walk TEMP till TEMP.next == NULL;  TEMP.next = N    [O(n)]
```

```
INSERT_POS:    Walk to pos-1;  N.next = TEMP.next;  TEMP.next = N    [O(n)]
```

```
DELETE_BEGIN:  TEMP = HEAD;  HEAD = HEAD.next;  Delete TEMP    [O(1)]
```

```
DELETE_END:    Walk till TEMP.next.next == NULL;  
               Delete TEMP.next;  TEMP.next = NULL    [O(n)]
```

```
DELETE_POS:    Walk to pos-1;  DEL = TEMP.next;  
               TEMP.next = DEL.next;  Delete DEL    [O(n)]
```

```
SEARCH:        TEMP = HEAD, pos = 0;  
               While TEMP != NULL: check data, TEMP = TEMP.next, pos++    [O(n)]
```

◆ DOUBLY LINKED LIST

```

INSERT_BEGIN:  N.prev = NULL;  N.next = HEAD;
               If HEAD != NULL → HEAD.prev = N;  HEAD = N    [O(1)]

INSERT_END:    Walk to last;  TEMP.next = N;  N.prev = TEMP;  N.next = NULL    [O(n)]

INSERT_POS:    Walk to pos-1;  N.next = TEMP.next;  N.prev = TEMP;
               If TEMP.next != NULL → TEMP.next.prev = N;  TEMP.next = N    [O(n)]

DELETE_BEGIN:  TEMP = HEAD;  HEAD = HEAD.next;
               If HEAD != NULL → HEAD.prev = NULL;  Delete TEMP    [O(1)]

DELETE_END:    Walk to last (TEMP.next == NULL);
               TEMP.prev.next = NULL;  Delete TEMP    [O(n)]

DELETE_POS:    Walk to pos;  TEMP.prev.next = TEMP.next;
               If TEMP.next != NULL → TEMP.next.prev = TEMP.prev;  Delete TEMP    [O(n)]

```

◆ STACK

```

PUSH(X):  If TOP == MAX-1 → Overflow
          Else: TOP = TOP + 1;  STACK[TOP] = X    [O(1)]

POP:      If TOP == -1 → Underflow
          Else: X = STACK[TOP];  TOP = TOP - 1;  Return X    [O(1)]

```

◆ QUEUE

```

ENQUEUE(X):  If REAR == MAX-1 → Overflow
             If FRONT == -1 → FRONT = 0
             REAR = REAR + 1;  QUEUE[REAR] = X    [O(1)]

DEQUEUE:     If FRONT == -1 OR FRONT > REAR → Underflow
             X = QUEUE[FRONT]
             If FRONT == REAR → FRONT = REAR = -1
             Else FRONT = FRONT + 1;  Return X    [O(1)]

ISFULL:      If REAR == MAX-1 → TRUE  else FALSE    [O(1)]
ISEMPTY:     If FRONT == -1 OR FRONT > REAR → TRUE  else FALSE    [O(1)]

```

◆ TREE TRAVERSALS

```

INORDER(node):  If NULL return
                INORDER(left);  Print data;  INORDER(right)    [O(n)]

PREORDER(node):  If NULL return
                Print data;  PREORDER(left);  PREORDER(right)    [O(n)]

POSTORDER(node):  If NULL return
                 POSTORDER(left);  POSTORDER(right);  Print data    [O(n)]

```

◆ BST

```

BST_INSERT(X):  Create N;  If Root == NULL → Root = N;
                Walk: X < data → go left;  X > data → go right;
                Plant N at NULL spot    [O(log n) avg, O(n) worst]

BST_SEARCH(X):  Walk: X == data → Found;  X < data → left;  X > data → right
                If NULL → Not Found    [O(log n) avg, O(n) worst]

BST_DELETE(X):  Leaf → delete;  1 child → bypass;
                2 children → find inorder successor, swap, delete successor

```

◆ DFS & BFS

```

DFS(G, S):  Create STACK, VISITED[]
            Push S
            While STACK not empty:
                V = POP
                If not visited: mark visited, Print V
                Push all unvisited neighbors of V    [O(V+E)]

BFS(G, S):  Create QUEUE, VISITED[]
            Mark S visited;  ENQUEUE S
            While QUEUE not empty:
                V = DEQUEUE;  Print V
                For each unvisited neighbor W:
                    Mark W visited;  ENQUEUE W    [O(V+E)]

```

◆ QUICK SORT

```

PARTITION(A, low, high):
    pivot = A[high];  i = low - 1
    For j = low to high-1:
        If A[j] <= pivot:  i++;  Swap A[i] and A[j]
    Swap A[i+1] and A[high]
    Return i+1

QUICKSORT(A, low, high):
    If low < high:
        pi = PARTITION(A, low, high)
        QUICKSORT(A, low, pi-1)
        QUICKSORT(A, pi+1, high)

Complexity: O(n log n) avg | O(n²) worst | Space: O(log n) | In-place: YES | Stable: NO

```

◆ MERGE SORT

```

MERGE(A, left, mid, right):
    L = A[left..mid];  R = A[mid+1..right]
    i = 0,  j = 0,  k = left
    While i < n1 AND j < n2:
        If L[i] <= R[j]: A[k] = L[i];  i++
        Else: A[k] = R[j];  j++
        k++
    Copy remaining L or R elements

```

```

MERGESORT(A, left, right):
    If left < right:
        mid = (left + right) / 2
        MERGESORT(A, left, mid)
        MERGESORT(A, mid+1, right)
        MERGE(A, left, mid, right)

```

Complexity: $O(n \log n)$ ALWAYS | Space: $O(n)$ | In-place: NO | Stable: YES

◆ INFIX TO POSTFIX & POSTFIX EVALUATION

```

INFIX_TO_POSTFIX:
    Operand      → Output directly
    '('          → Push
    ')'          → Pop until '(' (discard '(')
    Operator     → Pop higher/equal precedence first, then Push
    End of input → Pop all remaining

EVALUATE_POSTFIX:
    Number       → Push
    Operator     → Pop B (top), Pop A (below), Compute A op B, Push result
    End         → Pop final result

Precedence: ^ (3, right-assoc) > * / (2, left-assoc) > + - (1, left-assoc)

```

► MASTER COMPLEXITY TABLE

◆ DATA STRUCTURES

Operation	Array	SLL	DLL	Stack	Queue
Insert Begin	$O(n)$	$O(1)$	$O(1)$	$O(1)$ Push	$O(1)$ Enqueue
Insert End	$O(1)^*$	$O(n)$	$O(n)$	—	$O(1)$
Insert Middle	$O(n)$	$O(n)$	$O(n)$	—	—
Delete Begin	$O(n)$	$O(1)$	$O(1)$	$O(1)$ Pop	$O(1)$ Dequeue
Delete End	$O(1)^*$	$O(n)$	$O(n)$	—	—
Delete Middle	$O(n)$	$O(n)$	$O(n)$	—	—
Search	$O(n)$	$O(n)$	$O(n)$	$O(n)$	$O(n)$
Access by index	$O(1)$	$O(n)$	$O(n)$	$O(1)$ TOP	$O(1)$ FRONT

◆ SORTING & SEARCHING ALGORITHMS

Algorithm	Best	Average	Worst	Space
Quick Sort	$O(n \log n)$	$O(n \log n)$	$O(n^2)$	$O(\log n)$
Merge Sort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(n)$
Linear Search	$O(1)$	$O(n)$	$O(n)$	$O(1)$

BST Search	$O(\log n)$	$O(\log n)$	$O(n)$ skewed	$O(1)$
DFS / BFS	$O(V+E)$	$O(V+E)$	$O(V^2)$ adj matrix	$O(V)$
Tree Traversals	$O(n)$	$O(n)$	$O(n)$	$O(h)$ h =height

► MASTER DIFFERENCES TABLES

◆ Array vs Linked List

Feature	Array	Linked List
Memory	Contiguous	Non-contiguous
Size	Fixed (static)	Dynamic
Access	$O(1)$ random	$O(n)$ sequential
Insertion	$O(n)$ shifting needed	$O(1)$ at beginning
Memory waste	Possible (fixed size)	None (allocated as needed)
Extra memory	None	Pointer per node

◆ SLL vs DLL

Feature	Singly Linked List	Doubly Linked List
Pointers per node	1 (next only)	2 (prev + next)
Traversal direction	Forward only	Both directions
Memory per node	Less	More
Delete at end	Stop at second-to-last (TEMP.next.next == NULL)	Stop at last (TEMP.next == NULL), use TEMP.prev
Delete at position	Stop at pos-1, use DEL pointer	Stop at pos, use TEMP.prev directly
Insert at middle	2 pointer updates	4 pointer updates
Complexity (all ops)	Same as DLL	Same as SLL

◆ Stack vs Queue

Feature	Stack	Queue
Principle	LIFO (Last In, First Out)	FIFO (First In, First Out)
Insert end	TOP (one end only)	REAR
Delete end	TOP (same end)	FRONT
Pointers	One (TOP)	Two (FRONT, REAR)
Empty condition	TOP == -1	FRONT == -1 OR FRONT > REAR

Full condition	TOP == MAX-1	REAR == MAX-1
Operations	Push, Pop	Enqueue, Dequeue
Application	Undo, recursion, expression eval	Scheduling, BFS, print spooling

◆ DFS vs BFS

Feature	DFS	BFS
Strategy	Deep first	Wide / Level first
Data Structure	Stack (LIFO)	Queue (FIFO)
Shortest path?	✗ No	✓ Yes (unweighted)
Memory	Less	More
Visited marking	When POPPED	When ENQUEUED
Time Complexity	$O(V+E)$	$O(V+E)$
Best for	Maze, cycles, topological sort	Shortest path, social networks

◆ Quick Sort vs Merge Sort

Feature	Quick Sort	Merge Sort
Worst Case	$O(n^2)$	$O(n \log n)$
Space Complexity	$O(\log n)$	$O(n)$
In-place?	✓ Yes	✗ No
Stable?	✗ No	✓ Yes
Divide method	Unequal (pivot-based)	Equal halves (mid)
Main work in	Divide (partition)	Conquer (merge)
Worst case input	Already sorted array	None (always $n \log n$)
Best for	Average case, arrays, memory limited	Guaranteed perf, linked lists, stability

◆ Binary Tree vs BST

Feature	Binary Tree	Binary Search Tree
Order property	None	Left < Root < Right
Search	$O(n)$ — must check all	$O(\log n)$ average
Inorder result	Random order	Always sorted (ascending)
Duplicates	Allowed	Usually not allowed
Use	General hierarchy	Efficient search/sort

◆ Infix vs Prefix vs Postfix

Feature	Infix	Prefix	Postfix
	Between	Before	After

Operator position			
Parentheses	Required	Not needed	Not needed
Human readable	Yes	Hard	Hard
Computer eval	Complex	Simple	Simplest
Evaluation order	With precedence	Right to Left	Left to Right
Stack used for	Operators (conversion)	Operands (evaluation)	Operands (evaluation)

► MOST PROBABLE EXAM QUESTIONS — UBIT DSA FINAL

☆☆☆ HIGHEST PROBABILITY (Will almost certainly appear)

1. Write the INSERT and DELETE algorithms for arrays with dry run.
2. Write all SLL insertion/deletion algorithms with one dry run each.
3. Difference between SLL and DLL with algorithms showing the extra pointer steps.
4. Write PUSH and POP algorithms. Explain Overflow and Underflow.
5. Write ENQUEUE and DEQUEUE algorithms with dry run.
6. Perform Inorder, Preorder, Postorder traversal on a given tree.
7. Insert given elements into BST and draw the tree. Then perform Inorder.
8. Write DFS and BFS algorithms. Perform dry run on same graph. Compare.
9. Write QUICKSORT and PARTITION algorithms. Perform dry run.
10. Write MERGESORT and MERGE algorithms. Perform dry run.
11. Convert infix expression to postfix. Show full stack trace.
12. Evaluate a postfix expression using stack. Show each step.
13. Difference between Stack and Queue.
14. Difference between Quick Sort and Merge Sort.
15. Why DFS uses Stack and BFS uses Queue — explain logically.

☆☆ MEDIUM PROBABILITY (Likely to appear)

1. What is a graph? Types of graphs with examples.
2. Adjacency Matrix vs Adjacency List — draw both for a given graph.
3. Explain three cases of BST deletion with examples.
4. What is a Binary Tree? Types of binary trees.
5. Significance of Inorder traversal in BST.
6. What is Divide and Conquer? How does Quick Sort / Merge Sort apply it?
7. Worst case of Quick Sort — when does it occur and how to avoid?
8. Why Merge Sort is always $O(n \log n)$?
9. What is stable sorting? Is Merge Sort stable? Is Quick Sort?
10. Expression tree — what traversal gives infix, prefix, postfix?
11. Difference between prefix and postfix evaluation.

☆ GOOD TO KNOW (May appear as bonus/extra)

1. What is Circular Queue? Why is it needed?
2. Applications of Stack (5 points).
3. Applications of Queue (5 points).
4. Applications of DFS and BFS.
5. Why does Merge Sort need extra $O(n)$ space? What is in-place sorting?
6. Reconstruct a tree from Inorder and Preorder sequences.
7. Time complexity of BST operations — when is worst case $O(n)$?
8. What is a skewed tree? How does it affect BST performance?
9. Graph terminology — define all terms with diagram.

► LAST-MINUTE CHEAT SHEET

DATA STRUCTURE KEY CONDITIONS

Array Insert: invalid if $\text{pos} > n$ (can insert AT end, $\text{pos} == n$ is valid)
Array Delete: invalid if $\text{pos} > n$ (cannot delete at n , last valid is $n-1$)
Stack full: $\text{TOP} == \text{MAX}-1$ Stack empty: $\text{TOP} == -1$
Queue full: $\text{REAR} == \text{MAX}-1$ Queue empty: $\text{FRONT} == -1$ OR $\text{FRONT} > \text{REAR}$
SLL/DLL empty: $\text{HEAD} == \text{NULL}$

SLL vs DLL KEY DIFFERENCES

SLL Delete End: stop at SECOND-TO-LAST ($\text{TEMP.next.next} == \text{NULL}$)
DLL Delete End: stop at LAST node ($\text{TEMP.next} == \text{NULL}$), use TEMP.prev

SLL Delete Pos: stop at $\text{pos}-1$, need DEL pointer
DLL Delete Pos: stop at pos itself, use TEMP.prev directly

DLL Insert Begin: extra step $\rightarrow \text{HEAD.prev} = \text{N}$
DLL Insert End: extra step $\rightarrow \text{N.prev} = \text{TEMP}$
DLL Insert Mid: 4 pointers (N.next , N.prev , next.prev , TEMP.next)

SORTING KEY FACTS

Quick Sort worst case: sorted array + last-element pivot = $O(n^2)$
Merge Sort: ALWAYS $O(n \log n)$, NEVER degrades
Quick Sort: in-place YES, stable NO, space $O(\log n)$
Merge Sort: in-place NO ($O(n)$ space), stable YES

TRAVERSALS

INorder	= IN middle	= L-Root-R = BST gives SORTED output	= Infix expression
PREorder	= PRE first	= Root-L-R = copy tree	= Prefix expression
POSTorder	= POST last	= L-R-Root = delete tree	= Postfix expression

GRAPH TRAVERSAL

DFS = STACK = DEEP = mark when POPPED = does NOT find shortest path
BFS = QUEUE = WIDE = mark when ENQUEUED = FINDS shortest path (unweighted)

EXPRESSION EVALUATION

Postfix eval: scan LEFT to RIGHT

number → push

operator → pop B (top), pop A (below B), compute A op B, push result

Prefix eval: scan RIGHT to LEFT

number → push

operator → pop A (top = left), pop B (right), compute A op B, push result

Infix → Postfix:

operand → output directly

(→ push always

) → pop till matching ((discard the (

operator → pop higher/equal precedence from stack, then push

end → pop all remaining

Precedence: ^ (highest, right-assoc) > * / (mid, left-assoc) > + - (lowest, left-assoc)

COMPLEXITY QUICK REFERENCE

O(1): Array access by index, Stack push/pop, Queue enqueue/dequeue,
SLL/DLL insert at beginning, SLL/DLL delete from beginning

O(n): Array insert/delete/search, SLL/DLL most operations, Linear search

O(log n): BST search/insert average, Binary search

O(n log n): Quick Sort (best/avg), Merge Sort (ALWAYS)

O(n²): Quick Sort worst case, Bubble/Selection/Insertion sort

O(V+E): DFS, BFS (with adjacency list)

O(V²): DFS, BFS (with adjacency matrix)

► FINAL EXAM DAY STRATEGY

📅 Before the Exam (Night Before):

- Read the Last-Minute Cheat Sheet 3 times
- Practice writing one dry run each of: Infix→Postfix, QuickSort Partition, BFS
- Sleep well — tired brain makes silly mistakes

📁 During the Exam:

1. **Read ALL questions first** — identify which ones you're most confident in
2. **Start with your strongest topic** — builds confidence and gets early marks
3. **For every algorithm:** Write heading, Input/Output, numbered steps, Complexity
4. **For every dry run:** Draw diagrams, make tables, show every single step
5. **For theory questions:** 3-4 well-organized paragraphs with key terms **bolded**
6. **For difference questions:** Always use a TABLE format — looks professional, scores full
7. **Time management:** Don't spend more than 15 minutes on any single question
8. **At the end:** Check overflow/underflow conditions in algorithms — easy marks lost here

☆☆ Magic Words Examiners Love:

- "LIFO / FIFO principle"
- "O(n) time complexity"

- "Base case for recursion"
- "Overflow / Underflow condition"
- "Inorder traversal of BST gives sorted output"
- "DFS uses Stack, BFS uses Queue"
- "Pivot placed at its FINAL correct position"
- "Merge Sort requires $O(n)$ extra space because..."
- "The left subtree contains values less than root, right subtree contains values greater"

🏆 **YOU ARE FULLY PREPARED!** 🏆

All 14 Topics Covered — Theory + Algorithms + Dry Runs + Exam Q&A +
Tricks

**Arrays • SLL • DLL • Stack • Queue • Trees • BST • Tree Traversal
Graphs • DFS • BFS • Quick Sort • Merge Sort • Infix/Prefix/Postfix**

✨ **Go get that 4.0 GPA! You've got this!** ✨